Information Fusion 132 (2026) 104241



Contents lists available at ScienceDirect

Information Fusion

journal homepage: www.elsevier.com/locate/inffus





Security and privacy in LLMs: A comprehensive survey of threats and mitigation strategies

Aymen Dia Eddine Berini [a], Norziana Jamil [a,*], Ala-Eddine Benrazek [b,c], Abderrahmane Lakas [d], Leila Ismail [e], Mohamed Amine Ferrag [d], Kwok-Yan Lam [f]

[a] Department of Information Systems and Security, College of Information Technology, United Arab Emirates University, Al Ain, 15551, United Arab Emirates
[b] Department of Computer Science, CSAAI-Lab Laboratory, Ziane Achour University, Djelfa, 17000, Algeria
[c] Labstic Laboratory, 8 Mai 1945 University, Guelma, 24000, Algeria
[d] Department of Computer and Network Engineering, College of Information Technology, United Arab Emirates University, Al Ain, 15551, United Arab Emirates
[e] Department of Computer Science and Software Engineering, College of Information Technology, United Arab Emirates University, Al Ain, 15551, United Arab Emirates
[f] School of Computer Science and Engineering, Nanyang Technological University, Singapore, 639798, Singapore

ARTICLE INFO

Keywords:
Large language models
Security
Privacy
Adversarial attacks
LLM threats
Defense mechanisms

ABSTRACT

Large Language Models (LLMs) have transformed natural language processing and are increasingly utilized in essential sectors such as healthcare, financial services, and education. However, their adoption has raised significant security and privacy concerns due to their vulnerability throughout different lifecycle phases to several attacks, including data poisoning, model inversion, prompt injection, and supply chain vulnerabilities. Although previous studies have investigated these risks, current surveys often lack sufficient comprehensiveness, technical rigor, or lifecycle consideration. To address this gap, we present a lifecycle-oriented, and in-depth analysis of LLM security and privacy. We introduce a taxonomy of threats spanning data-centric, model-centric, and deployment and interaction-centric vectors, each mapped to corresponding defense mechanisms. We also address cross-cutting risks, including hallucinations, algorithmic bias, and vulnerable plugin interfaces. In addition, we evaluate the efficacy and limitations of existing defenses, incorporate new security tools and assessment frameworks, and situate our findings within the broader literature through extensive comparative analysis. Finally, we identify open research challenges and outline future directions for developing secure, robust, and regulation-compliant LLM systems. This survey serves as a foundational reference for researchers, developers, and policymakers in understanding the evolving adversarial landscape of LLMs.

1. Introduction

LLMs have emerged as a transformative technology in artificial intelligence, demonstrating advanced capabilities in areas such as natural language comprehension, generation, reasoning, and following instructions. Their integration into critical domains such as healthcare, banking, law, and education has significantly improved human-computer interaction and enabled more autonomous decision-making [1].

However, the deployment of LLMs has also raised serious security and privacy issues. These issues are rooted in the very characteristics that make LLMs so powerful, specifically their ability to memorize and reproduce sensitive information, their training on vast and often uncurated data corpora, and their large-scale architectures [2].

Recent research indicates that LLMs, while highly useful, can also be exploited for malicious purposes such as prompt injection, model extraction, data leakage, or misinformation campaigns [3–6]. They are also vulnerable to threats during training, such as data poisoning and insertion of backdoors, as well as threats during inference, including membership inference and model evasion [7,8]. These risks are amplified as LLMs integrate with external tools and APIs, making the supply chain more susceptible.

Despite a growing body of literature on LLM safety, several limitations remain. Many studies examine security and privacy risks in isolation, without a comprehensive lifecycle framework covering all stages, from data collection and pre-training to fine-tuning, deployment, real-time inference, and maintenance [9,10]. Existing taxonomies and threat

* Corresponding author.
E-mail addresses: berini.aymen@uaeu.ac.ae (A.D.E. Berini), norziana@uaeu.ac.ae (N. Jamil), alaeddine.benrazek@univ-djelfa.dz (A.-E. Benrazek), alakas@uaeu.ac.ae (A. Lakas), leila@uaeu.ac.ae (L. Ismail), mohamed.ferrag@uaeu.ac.ae (M.A. Ferrag), kwokyan.lam@ntu.edu.sg (K.-Y. Lam).

https://doi.org/10.1016/j.inffus.2026.104241
Received 29 October 2025; Received in revised form 29 January 2026; Accepted 16 February 2026
Available online 19 February 2026
1566-2535/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

models are often too broad to capture the complex interplay between adversarial goals, system architectures, and real-world applications. Furthermore, while defense mechanisms such as differential privacy, federated learning, and guardrails have been proposed, their effectiveness is often limited and is not adequately evaluated in various threat models and deployment scenarios [11–13].

1.1. Contributions of this survey

With the widespread use of LLMs in many real-world applications, the necessity for a thorough and systematic comprehension of their security and privacy issues has increased. Despite earlier research on specific LLM vulnerabilities and mitigations, the literature remains fragmented. It often lacks a unified framework that addresses the entire lifecycle of LLM development and deployment. To address this gap, our survey provides a comprehensive and lifecycle-focused overview of current security and privacy studies in LLMs.

- *End-to-End Lifecycle-Centric Taxonomy of Threats*: We develop an organized and detailed threat taxonomy that covers the complete LLM lifetime, including pre-training, fine-tuning, deployment, and interaction stages. Threats are classified into three primary dimensions: data-centric, model-centric, and deployment/interaction-centric, along with an additional layer for cross-cutting vulnerabilities. This offers a comprehensive perspective on the adversarial environment impacting LLMs.
- *Systematic Mapping Between Threats and Defenses*: For each type of threat identified, we systematically examine and define the associated defense mechanisms. We evaluate the efficacy, assumptions, and limitations of these defenses in the context of their operational functions, thereby providing practitioners with a decision-making framework for implementing adaptive and layered safeguards.
- *Comprehensive Coverage of Cross-Cutting Threats*: In addition to phase-specific attacks, we also take into account problems that affect more than one phase, such as hallucinations, algorithmic bias, LLM supply chain weaknesses, and unsafe API output handling. These problems affect all stages of the life cycle and pose systemic risks that have not been adequately stressed in previous studies.
- *Structured Defense Strategy Framework*: Along with our threat categorization, we present a parallel defense taxonomy that includes data-centric, model-centric, and deployment/interaction-centric defenses. There are both proactive and reactive approaches to achieve this, including data sanitization, differential privacy, safe fine-tuning, output filtering, and adversarial red teaming.
- Comparative Literature Matrix: We develop a detailed comparison matrix that contrasts our study with existing surveys, highlighting differences in lifecycle coverage, threat granularity, defense mapping, and consideration of real-world deployment issues.
- Integration of security tools and evaluation frameworks: Synthesize recent advances in adversarial tools, LLM-specific security testbeds, and robustness evaluation frameworks. These tools are contextualized within the broader threat-defense landscape, highlighting their role in assessing LLM security posture.
- Identification of Open Research Challenges: Finally, we discuss unresolved issues and future directions, including the need for adaptive defensive mechanisms, robust alignment verification, scalable mitigation in multi-agent scenarios, and compliance-aware design within evolving AI regulatory frameworks.

Unlike prior surveys that focus either on specific threat categories, such as adversarial attacks [13] and jailbreaks [21], or on a high-level lifecycle breakdown [10], our work presents an integrated, phase-aware taxonomy that explicitly links each threat to its corresponding defensive measures and evaluates their practical effectiveness. This lifecycle perspective reveals, for example, that data poisoning defenses must be applied not only during pre-training but also during fine-tuning and retrieval-augmented generation stages, a nuance often overlooked in earlier reviews. Furthermore, our systematic mapping of defenses incorporates recent tools and evaluation frameworks, including security testbeds and red-teaming platforms, which enable reproducible robustness assessments and provide practitioners with an actionable roadmap for secure LLM deployment.

1.2. Methodology

In this survey, we follow a clear and structured approach to study the security of LLMs. Wepromp commence by examining the deployment environment, which refers to the real-world context in which the model is employed, such as in healthcare, finance, education, or software development. Next, we describe the system model, which explains the different parts of the LLM lifecycle, including data collection, pre-training, fine-tuning, inference, and how the model connects with external tools. We then set out the threat model, where we define what an attacker can do (for example, having black-box or white-box access), which part of the system they may target (data, model, or deployment), and what their goals might be (such as stealing data, inserting backdoors, or bypassing safety rules). After this, we consider the risks, focusing on how these threats may harm confidentiality, integrity, or availability of the system. To respond to these risks, we outline control mechanisms, such as data cleaning, differential privacy, safe fine-tuning, adversarial testing, and output filtering. Finally, we look at mitigation strategies, which show how these controls can reduce risks in practice, while also highlighting their strengths and weaknesses.

1.3. Paper organization

The overall structure of this paper is illustrated in Fig. 1, which outlines the main sections and their subsections. The paper is designed to provide a systematic exploration of the security and privacy landscape of LLMs, with each section addressing a specific layer of the threat ecosystem, along with corresponding defenses, tools, and evaluation frameworks.

Section 2 reviews prior literature on the security and privacy of LLMs. Section 3 establishes the conceptual foundation by defining essential architectural principles, training paradigms, and deployment contexts of LLMs. Section 4 presents our threat-modeling methodology. We adapt structured frameworks (e.g., STRIDE) to LLM systems and demonstrate how traditional categories (spoofing, tampering, information disclosure, denial of service, elevation of privilege) must be extended to capture LLM-specific vectors such as prompt injection, memorization-driven leakage, and data-poisoning during upstream training. Section 5 presents a key contribution: a lifecycle-aligned threat taxonomy that categorizes attacks into data-centric, model-centric, and deployment/interaction-centric classes, exemplified by data poisoning, training data leakage, model theft and inversion, prompt injection, and denial-of-service. It also extends the discussion to cross-cutting threats that span multiple lifecycle stages, such as hallucinations, algorithmic bias, supply chain vulnerabilities, and insecure API outputs. Section 6 presents related defense and mitigation tactics aligned with the threat taxonomy, including countermeasures at the data, model, and interaction levels. Section 7 examines persistent challenges and emerging research frontiers in trustworthy LLM deployment, while outlining research pathways for evolving threat landscapes within dynamic operational environments. Section 8 concludes the paper by summarizing essential ideas and emphasizing the significance of a lifecycle-aware perspective. This structure provides a methodical guide for understanding, evaluating, and mitigating the growing security and privacy issues associated with LLMs.

2. Related work

As LLMs have gained popularity, researchers have intensified efforts to document and evaluate their associated privacy and security issues.

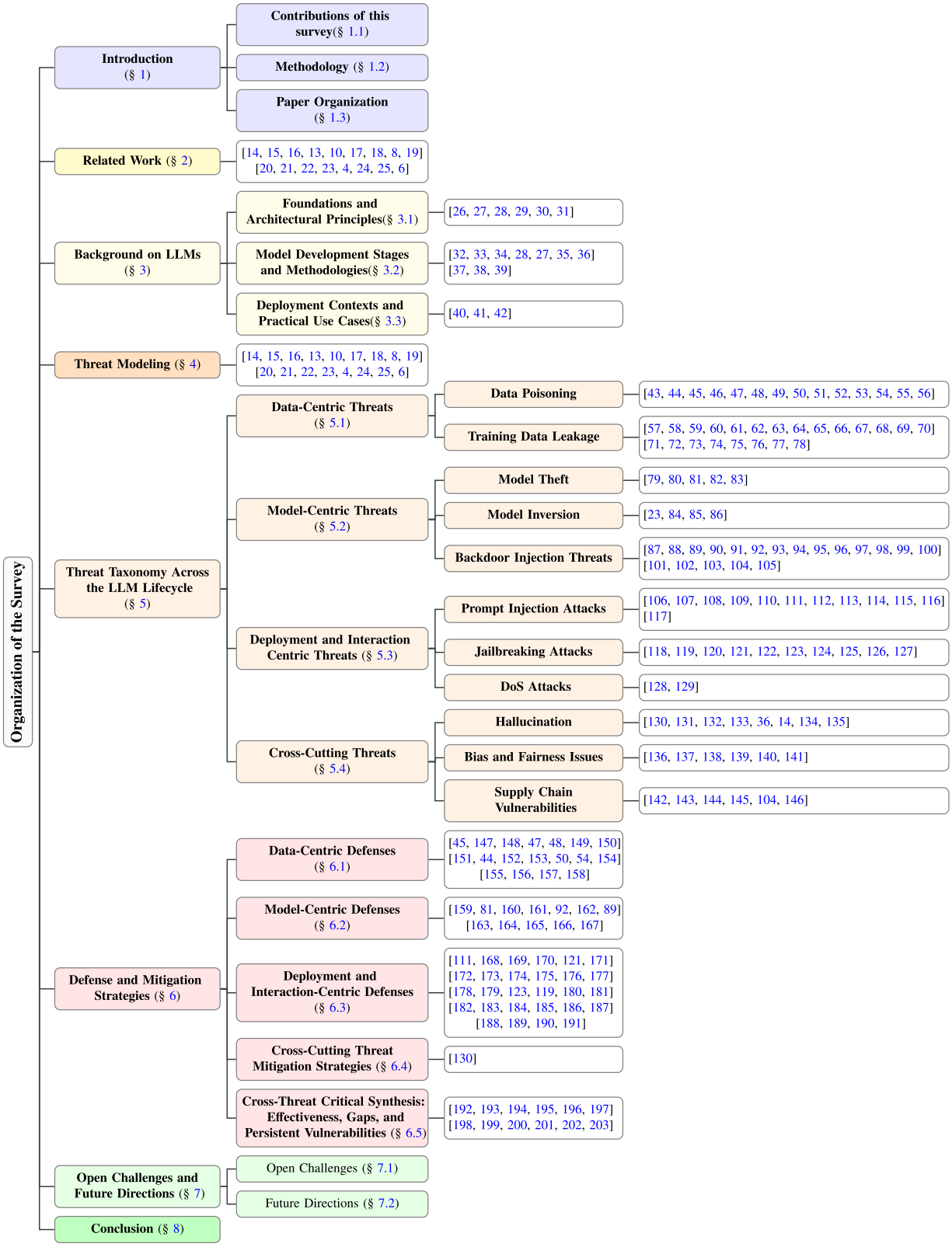


Fig. 1. The organization of the survey.

Although past surveys have provided valuable information, closer examination reveals significant gaps in the literature when it comes to scope, depth, and methodological consistency.

While existing surveys provide valuable insights into specific aspects of LLM security and privacy, they often lack a holistic, phase-aware perspective that systematically links threats to corresponding defenses across the entire model life cycle. For instance, Liu et al. [14] and Weidinger et al. [15] provide high-level overviews of LLM misuse and ethical implications but lack a technical dissection of attack vectors and defense mechanisms specific to each phase of the LLM life cycle. More

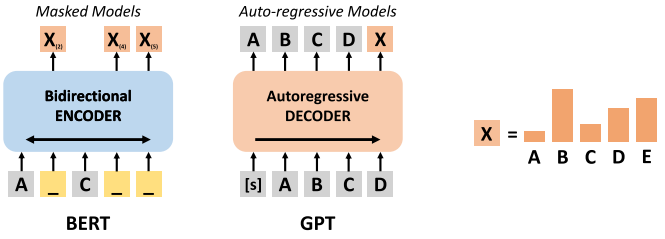


Fig. 2. Bidirectional and auto-regressive transformers.

recent studies, including Shayegani et al. [16] and Dong et al. [13], focus primarily on inference-time adversarial attacks (e.g., jailbreaks, prompt injection) but do not deeply examine upstream vulnerabilities such as data poisoning or supply-chain compromises. Wang et al. [10] propose a life-cycle-aware framework spanning five stages; yet, their analysis offers limited detail on cross-cutting risks (e.g., hallucinations, bias, plugin vulnerabilities) and does not provide a structured mapping between attack vectors and defense mechanisms. Similarly, Abdali et al. [17] and other surveys approaching the topic from a responsible AI perspective often underrepresent the technical execution of threats and the practical efficacy of mitigations. Works focusing on edge-based LLMs (e.g., Friha et al. [18]) extend the security discourse to novel deployment contexts; however, they do not articulate granular attack models or align defenses with evolving real-world threat landscapes.

In contrast, our survey adopts an end-to-end life-cycle taxonomy that categorizes threats into data-centric, model-centric, and deployment-/interaction-centric classes, each explicitly mapped to phase-appropriate defenses. Furthermore, we incorporate a dedicated analysis of cross-cutting threats that transcend individual life-cycle stages, and we integrate recent security tools and evaluation frameworks to support reproducible robustness assessment. Table 1 summarizes these comparative distinctions, highlighting how our work fills gaps in life-cycle coverage, types of attacks, defense techniques, evaluation metrics and tools, open issues, and future directions.

3. Background on large language models

3.1. Foundations and architectural principles

Advancements in LLMs have significantly driven the recent evolution of NLP [195]. These models are characterized by immense parameter counts and vast training datasets. They demonstrate strong performance across tasks ranging from text completion to sophisticated reasoning and decision-making. The underlying architecture of most contemporary LLMs is the Transformer, introduced by Vaswani et al. in 2017 [23], which has become the cornerstone of modern NLP due to its superior ability to capture long-range contextual dependencies.

The transformer architecture fundamentally relies on the self-attention mechanism, which computes relationships between all tokens in a sequence simultaneously. This enables efficient modeling of context regardless of positional distance. Unlike recurrent neural networks (RNNs) or convolutional neural networks (CNNs), transformers overcome inherent sequential processing limitations, thereby enabling parallel computation and significantly improved training efficiency on modern GPU hardware [24,25].

Early transformer-based models, such as Bidirectional Encoder Representations from Transformers (BERT) and Generative Pretrained Transformer (GPT), marked the beginning of a transformative shift in NLP (see Fig. 2). BERT introduced bidirectional training by masking tokens and predicting their identities from surrounding context, enhancing language understanding significantly [25]. In contrast, GPT adopted an autoregressive approach, predicting subsequent tokens in a sequence, optimizing for generative capabilities [196]. In 2019, Meta (formerly Facebook) introduced BART (Bidirectional and Auto-Regressive Transformers), which synthesized BERT's bidirectional context modeling with GPT's autoregressive generation through a denoising autoencoder framework [197]. Subsequently, the GPT family experienced significant architectural scaling. GPT-2 increased the number of parameters to 1.5 billion, while GPT-3 achieved a 116-fold increase to 175 billion parameters. This rapid growth greatly enhanced the models' ability to recognize subtle linguistic patterns and maintain long-range dependencies [24,26]. Subsequently, GPT-4 and LLaMA models extended this trend, demonstrating unprecedented generalization capabilities across numerous tasks and domains [27,28]. Most recently, GPT-5 has pushed these boundaries further, emphasizing not only greater scale and multimodality but also improved reasoning, safety alignment, and adaptability across highly specialized domains. This makes large-scale foundation models the most important part of next-generation AI systems [198].

3.2. Model development stages and methodologies

LLM training spans several interdependent stages: data acquisition, preprocessing, pre-training, fine-tuning, and evaluation. Each satge significantly influences overall performance and real-world applicability.

3.2.1. Data acquisition and preprocessing

Training datasets primarily include large-scale textual corpora sourced from web crawls, digital libraries, scientific repositories, Wikipedia, and domain-specific collections [29]. Data preprocessing includes tokenization (e.g., Byte Pair Encoding [BPE], SentencePiece), data cleaning, normalization, and bias mitigation, ensuring robust model performance and generalization [30,31].

3.2.2. Pre-training methods

LLMs typically employ either masked language modeling (MLM), as in BERT, or causal language modeling (CLM), as in GPT. MLM predicts randomly masked tokens within contexts, enhancing contextual bidirectional language comprehension. Conversely, CLM sequentially predicts tokens based on preceding contexts, enhancing text generation capabilities [24,25]. Hybrid approaches such as XLNet combine permutations with autoregressive training to improve model performance by capturing contextual dependencies more effectively [32].

3.2.3. Fine-tuning and alignment approaches

Supervised Fine-Tuning (SFT) adapts pretrained LLMs to specific downstream tasks using task-specific datasets, significantly improving application-specific performance [33]. Reinforcement Learning from Human Feedback (RLHF) further refines outputs by aligning model responses with human preferences and ethical standards through iterative human-guided training, greatly enhancing model alignment and safety [33].

3.2.4. Evaluation techniques and metrics

LLM evaluation utilizes metrics such as perplexity (for language modeling uncertainty), BLEU (translation), ROUGE (summarization), and standardized benchmarks including GLUE, SuperGLUE, and Massive Multitask Language Understanding (MMLU) [34,35]. Comprehensive benchmarks increasingly consider ethical implications, bias, robustness, and interpretability, addressing broader societal deployment concerns [36].

3.3. Deployment contexts and practical use cases

LLMs have demonstrated significant impacts across diverse applications, including cybersecurity, healthcare, finance, education, customer service, and the legal industry.

Table 1
Comparative summary of existing surveys on LLM security and privacy.

Survey	Year	Key Contribution	Application	Type of Attacks	Defense Techniques	Evaluation Metrics/Tools	Open Issues	Future Directions
[15]	2022	Developed a comprehensive taxonomy of ethical and social risks associated with LMs, categorized into six areas.	○	◐	◐	○	○	○
[14]	2023	Comprehensive survey and evaluation framework for assessing LLM trustworthiness across seven dimensions to guide alignment and responsible deployment.	●	●	◐	◐	◐	○
[16]	2023	Surveys adversarial attacks on LLMs, categorizing attack types, analyzing vulnerabilities, and presenting defenses to support trustworthy AI.	○	●	●	●	○	○
[17]	2024	Investigates LLM security/privacy, vulnerabilities, misuse harms, mitigation strategies, and future research for enhanced risk management.	○	●	●	○	○	●
[18]	2024	Security/privacy in LLM-based edge intelligence systems.	○	●	◐	●	●	●
[8]	2024	First survey to systematically categorize LLM impacts on security into "The Good, The Bad, and The Ugly"; examines applications, threats, and defenses.	○	◐	◐	○	○	●
[19]	2024	Systematic review of backdoor attacks in LLMs used in communication networks; proposes novel attack taxonomy and identifies defense challenges.	○	◐	○	○	○	●
[20]	2024	Detailed taxonomy and analysis of jailbreak attacks and corresponding defenses, including evaluation methods.	○	●	●	●	○	○
[21]	2024	Introduced a module-oriented risk taxonomy for LLM systems, covering input, model, toolchain, and output modules	○	●	●	●	○	●
[22]	2024	Comparative analysis of adversarial attacks on LLMs with systematic categorization and implications	○	●	◐	○	●	●
[13]	2024	Presents a detailed review of guardrail (safeguard) mechanisms in LLMs including types, evaluation techniques, and toolkits.	○	●	●	◐	○	○
[23]	2024	Investigating data privacy threats in LLMs, assessing protection mechanisms, their limitations, and future directions for enhanced privacy.	○	●	●	○	◐	◐
[4]	2024	The paper examines LLM vulnerabilities to adversarial attacks, using ChatGPT as an example, analyzing differences from traditional ML, attack objectives, risk assessment, and open challenges.	○	◐	◐	○	◐	○
[10]	2024	Focuses on unique security/privacy threats across five LLM lifecycle phases; includes federated learning, unlearning, watermarking.	○	●	●	○	○	○
[24]	2025	Comprehensive survey of security/privacy attacks and mitigation strategies in LLMs; categorized attack vectors, future research roadmap.	○	●	●	◐	●	●
[25]	2025	Focused analysis of robustness challenges in LLMs; structured classification of robustness failures and mitigation strategies.	○	◐	●	◐	◐	◐
[6]	2025	Comprehensive survey analyzing privacy risks, leakage mechanisms, and mitigation strategies for securing and preserving privacy in LLMs.	◐	●	●	◐	●	●
This survey	/	Survey on LLM security and privacy, presenting a multidimensional threat-defense taxonomy, integration of security tools, and a comparative analysis of existing literature with clearly identified research gaps.	●	●	●	●	●	●

(●) : Yes (○) : No (◐) : Partial

- **Cybersecurity:** Domain-specific LLMs are increasingly deployed for vulnerability detection, malware classification, phishing detection, incident response automation, and cyber threat intelligence analysis. By learning from large-scale security logs and threat reports, they enhance intrusion detection and adaptive defense capabilities, though risks such as model exploitation and adversarial prompting remain active research challenges [199].
- **Software Engineering:** LLMs such as Codex and GitHub Copilot have transformed software development practices by providing intelligent assistance in code generation, debugging, and software maintenance. They suggest contextually relevant code snippets, improve developer productivity, and enable natural language interfaces for programming, making development accessible to non-experts [200].
- **Healthcare:** Models like Med-PaLM and BioGPT aid clinical decision-making, diagnostics, patient summarization, and biomedical literature analysis, significantly enhancing medical NLP applications [37,38].
- **Finance:** BloombergGPT exemplifies LLM application in financial NLP, enhancing market sentiment analysis, fraud detection, and regulatory compliance monitoring [39].

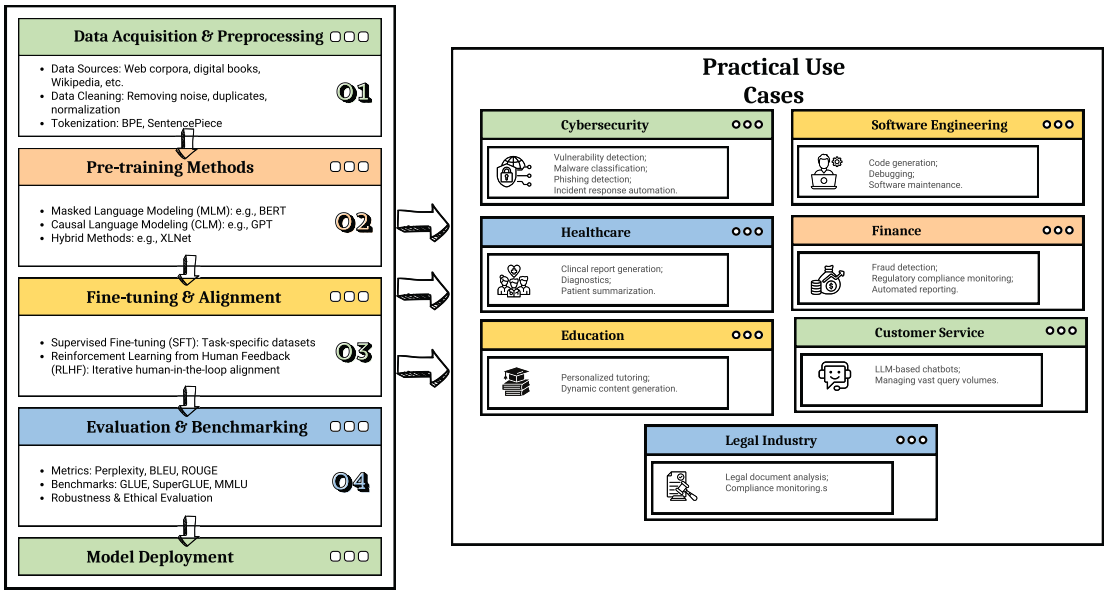


Fig. 3. Model development stages and domain-specific use cases of LLMs.

- **Education:** LLM-powered educational platforms support automated grading, personalized tutoring, and dynamic content generation, significantly enhancing learning personalization and scalability [201].
- **Customer Service:** Deployment of LLM-based chatbots improves customer interaction efficiency, managing vast query volumes with rapid, context-aware responses [202].
- **Legal Industry:** LLMs facilitate advanced legal document analysis, compliance monitoring, and litigation support, considerably boosting legal operational efficiency [203].

The rapid evolution of LLMs has given rise to a diverse ecosystem of architectures, capabilities, and deployment paradigms, spanning both open-source and proprietary offerings. These models vary significantly in terms of parameter scale, training objectives, data sources, alignment strategies, and intended use cases. Fig. 3 illustrates how LLMs are deployed across various computational contexts and aligned with practical applications in diverse domains. Table 2 provides a comprehensive comparison of recent and widely adopted LLMs.

4. Threat modeling

Threat modeling is a structured methodology that allows practitioners and researchers to identify possible adversarial actions, system vulnerabilities, and attack vectors that may compromise the confidentiality, integrity, availability, or ethical use of systems [2]. In the context of LLMs, it is crucial to engage in threat modeling, given the unpredictable and context-sensitive characteristics of the outputs they generate. Unlike deterministic systems, LLMs demonstrate dynamic behaviors, which require the anticipation of emergent risks that are often less predictable. The primary goals of threat modeling are to recognize assets that require safeguarding, to examine potential threat scenarios, to develop suitable appropriate countermeasures, and to enhance security strategies in response to evolving attack vectors [204].

The process begins with a clear definition of system boundaries, encompassing the model, its training datasets, exposed APIs, and any external integrations, as well as the flow of information between users and the underlying architecture. Once the scope is established, the next step is to identify critical assets requiring protection. These assets typically include sensitive user queries and outputs, the model's integrity, and the infrastructure supporting its operation, such as databases, APIs, and cloud services. In sensitive domains like healthcare or finance, safeguarding regulated data takes on particular urgency [205].

After identifying the assets, structured frameworks such as STRIDE can guide the threat analysis process when adapted to the LLM context [206]. Spoofing may involve adversaries impersonating legitimate users to manipulate outputs. Tampering could include altering model weights, training datasets, or API configurations. Information disclosure risks emerge when sensitive training or user data is inadvertently revealed in generated responses. Denial-of-service attacks may overwhelm LLM APIs with adversarial or excessive queries, while elevation of privilege arises when attackers exploit integrations to gain unauthorized access. Beyond these established categories, LLMs introduce new risks such as prompt injection, adversarial input manipulation, data poisoning, and unintended data leakage, which collectively expand the adversarial surface compared to conventional systems.

Mitigation strategies must address these vulnerabilities at both technical and operational levels. Input validation and contextual sanitization are critical for defending against prompt injection [207]. Data leakage risks can be reduced with differential privacy, strict access controls, and vigilant monitoring of outputs. Cryptographic integrity checks help prevent tampering, while resilient infrastructure combined with rate limiting and anomaly detection strengthens protection against denial-of-service attacks. The most effective approach is layered defense, ensuring redundancy and resilience across multiple threat categories.

Given the rapid evolution of both LLMs and adversarial techniques, threat models cannot remain static. They must be treated as dynamic frameworks, validated continuously through penetration testing, adversarial simulations, and red-teaming exercises [208]. Automated tools such as the Microsoft Threat Modelling Tool or OWASP Threat Dragon can aid systematic analysis and visualization, but human expertise remains indispensable for capturing contextual nuances. Iterative updates to the threat model should occur whenever new attack vectors are identified or significant system modifications are made [209]. This dynamic process is cyclical, beginning with system scoping and asset identification, progressing through threat analysis and mitigation, and returning regularly to validation and reassessment.

A practical example can be seen in the deployment of an LLM-powered chatbot for an e-commerce platform. The system faced risks including prompt injection attacks aimed at retrieving backend data, the unintended leakage of customer payment information, and API misuse designed to degrade performance. Through proactive threat modelling,

Table 2
Comparison of recent LLMs as of september 2025.

Model	Developer	Release Date	Parameters	Architecture	Context Length	GPQA Diamond	SWE-Bench	Humanity's Last Exam	Open Source
GPT-5	OpenAI	Aug 2025	N/A	Transformer-based, multimodal	272K tokens	87.3%	74.9%	25.32	No
Grok 4	xAI	Jul 2025	N/A	Transformer-based, multimodal	256K tokens	87.5%	75%	25.4	No
Gemini 2.5 Pro	Google	Mar 2025	N/A	Transformer-based, multimodal	1M tokens	86.4%	59.6%	21.6	No
Claude 4 Opus	Anthropic	May 2025	N/A	Transformer-based, multimodal	200K tokens	79.6%	72.5%	10.7	No
Claude 4 Sonnet	Anthropic	Mar 2025	N/A	Transformer-based, multimodal	200K tokens	75%	72.7%	7.7	No
Llama 4 Scout	Meta	Apr 2025	109B	Transformer-based	10M tokens	57.2%	–	4.3%	Yes
Llama 4 Maverick	Meta	No. 2024	400B	Transformer-based	1M tokens	69.8%	–	5.7	Yes
DeepSeek (R1-0528)	DeepSeek	May 2025	37B	Mixture of Experts (MoE) Tranformer	128K tokens	71.5%	49.2%	8.5	Yes
Qwen3-235B-A22B	Alibaba	Jul 2025	22B	Hybrid MoE	262K tokens	79%	–	11.8	Yes
Mistral Medium 3	Mistral AI	May 2025	N/A	Transformer-based	128K tokens	58.1%	–	4.52	No

the organization was able to anticipate these risks and apply countermeasures such as sensitive prompt filtering, output monitoring, and API rate limiting. These safeguards not only enhanced the technical robustness of the system but also reinforced user trust in the platform.

Ultimately, the value of threat modeling lies in its integration with broader best practices. Effective implementation requires collaboration among developers, security specialists, ethicists, and legal experts, ensuring that risks are addressed from both technical and societal perspectives. Advanced privacy-preserving methods, including homomorphic encryption, secure multiparty computation, and differential privacy, offer promising avenues for strengthening protection without compromising functionality. Regular reassessment and adversarial stress-testing through structured red-teaming further ensure that defenses remain adaptive to emerging threats.

In conclusion, threat modeling provides a structured and forward-looking roadmap for securing LLM applications. By embedding it into the development lifecycle, organizations can systematically identify, prioritize, and mitigate risks, striking a balance between innovation and security. This not only fosters the safe adoption of LLM technologies but also safeguards compliance, privacy, and user trust.

5. Threat taxonomy across the LLM lifecycle

This section analyzes the diverse threats and vulnerabilities inherent in LLM systems that can undermine their reliability, as categorized in the threat taxonomy presented in Fig. 4.

5.1. Data-centric threats

Data-centric threats in LLMs represent one of the most critical vectors of vulnerability, affecting the foundational integrity and trustworthiness of these systems. As LLMs are trained on vast corpora of heterogeneous, uncurated, and often opaque internet-scale datasets, the risk landscape surrounding data integrity, privacy, and authenticity becomes multifaceted and severe. These threats span the full model lifecycle, from pre-training to fine-tuning and real-time inference and compromise both security and ethical guarantees of generative AI systems. Fig. 5 illustrates the most common data-centric threats, including data poisoning and training data leakage.

5.1.1. Data poisoning

Data poisoning attacks are particularly insidious, as they exploit the openness of LLM training pipelines by injecting manipulated samples into the training corpus. These poisoned data entries can be crafted to subtly skew model behavior toward adversarial goals, such as embedding backdoors, introducing toxic associations, or biasing outputs without altering model performance on standard benchmarks. Given the scale of LLM training data, even low-rate poisoning (as low as 0.01% of the dataset) can produce measurable effects in target tasks or phrases, especially in fine-tuning or low-resource settings [40]. Data poisoning may be intentional, executed by malevolent individuals seeking to compromise the model, or unintentional, resulting from careless or uninformed data providers who disregard data quality and security standards.

Adversaries can employ multiple strategies to corrupt training datasets, thereby compromising the integrity and behavior of the resulting models. As outlined in [41], two notable poisoning techniques target commonly used web-sourced datasets. The first technique exploits discrepancies between what dataset curators observe during data collection and what end users eventually access. By acquiring a small number of domain names, an attacker can effectively manipulate the subset of data ingested by downstream users. The second technique involves tampering with periodically archived datasets, such as those found on Wikipedia. Here, attackers alter articles before they are included in snapshot repositories, ensuring that perturbed versions are

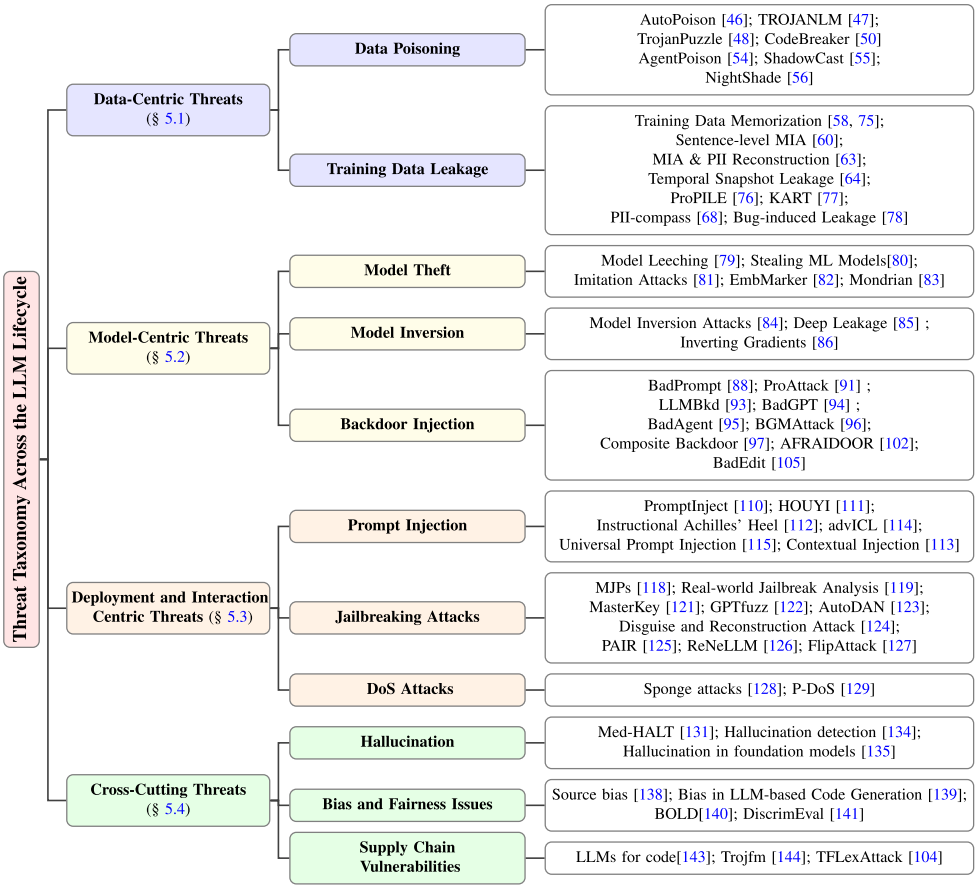


Fig. 4. Taxonomy of adversarial threats targeting LLMs.

Table 3
A comparative analysis of data poisoning attacks on LLMs.

Attack Name	Target	Technique	Trigger	Impact	Poisoning Stage	Stealthiness	Required Samples
AutoPoison [46]	Instruction-following LLMs (OracleLM)	Automated poisoning via adversarial context generation	Custom keyword insertion	Induces incorrect responses in target models	Pre-training/ Fine-tuning	Moderate	N/A (automated)
TROJANLM [47]	Host NLP systems	Trojan-injected models with deterministic triggers	Specially crafted inputs	Predictable model misbehavior	Pre-training	Low	Few
TrojanPuzzle [48]	Code generation models	Stealthy poisoning via distributed payloads	Code outside docstrings	Payload reconstructed during inference	Pre-training	High	Few
CodeBreaker [50]	CodeGen models	Fine-tuning with poisoned data	Malicious patterns in fine-tune set	Evades vulnerability detectors	Fine-tuning	High	Moderate (few %)
AgentPoison [54]	RAG-based LLM agents	Injection of poisoned documents into retrieval corpus	Query matches poisoned entry	Malicious responses via retrieval	Retrieval Corpus	High	Few poisoned docs
ShadowCast [55]	Vision-language models	Minimal-sample poisoning in training data	Visual-text prompts	Visually indistinguishable deviation	Training	High	50 samples
NightShade [56]	Text-to-image models	Prompt-specific poisoning during training	Prompt phrases	Indistinguishable manipulated outputs	Training	Very High	100 instances

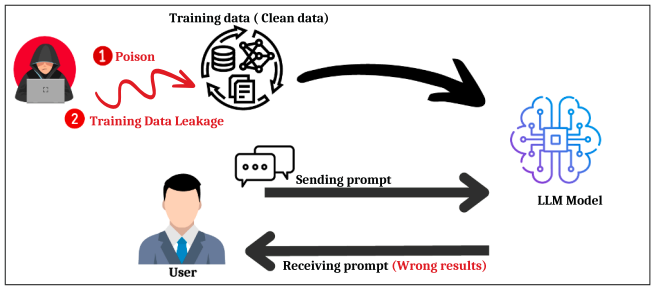


Fig. 5. Overview of threats to data.

permanently embedded within publicly available corpora. Wan et al. [42] conducted a focused examination of instruction-tuned language models, such as ChatGPT, uncovering a notable vulnerability to data poisoning attacks. Their study demonstrates that the insertion of a relatively small number of adversarially crafted training examples (approximately 100) can systematically induce erroneous behaviors across a wide range of tasks. This is particularly concerning given the heavy reliance of such models on large-scale, user-contributed datasets. A striking observation from their work is the emergence of an "Inverse Scaling" effect, whereby larger models exhibit increased susceptibility to these poisoning attempts. These insights underscore the imperative for stringent data quality controls and the development of resilient training pipelines, especially as instruction tuning becomes central to modern LLM development [42].

Further automation of such attacks has been demonstrated with *AutoPoison* [43], a framework that crafts adversarial contexts by modifying instructions in training data (typically through the inclusion of malicious keywords within model responses). Another variant, $TROJAN^{LM}$, proposed by Zhang et al. [44], introduces Trojan behavior into language

models. These models, once infected, exhibit deterministic misbehavior when specific triggers are encountered. Unlike traditional data poisoning methods that involve overt injection of malicious payloads into training samples, TrojanPuzzle [45] innovates by crafting stealthier inputs. It obfuscates malicious triggers within the data, such that the payload is reconstructed by the model during inference, even when placed outside of docstrings or comments.

Schuster et al. [46] initiated early research on the rising threat of data poisoning attacks, exposing critical vulnerabilities in ML supply chains, highlighting deeper concerns related to the security and ethical use of AI systems. Building on this, *CODEBREAKER* [47] demonstrates a more refined poisoning method applied during the fine-tuning of code generation models. Remarkably, it preserves core functionalities while remaining undetectable to conventional vulnerability scanners such as those examined in [48].

In recent developments, poisoning has also been extended to Retrieval-Augmented Generation (RAG) frameworks. RAG enhances LLM utility by integrating external knowledge sources, such as organizational databases, to facilitate domain-specific task execution [49,50]. However, works like AgentPoison [51] have shown that adversaries can manipulate RAG-based LLMs by inserting a limited number of poisoned documents into the retrieval corpus, thereby skewing the model's output.

Data poisoning threats are not confined to text-based models. Vision-language models (VLMs) also exhibit susceptibility. For instance, ShadowCast [52] demonstrates that inserting as few as 50 poisoned samples into training can alter the model's behavior without perceptible differences in the output, thus bypassing visual inspection. Similarly, NightShade [53] introduces prompt-specific poisoning attacks for text-to-image generative models, requiring only 100 perturbed samples to control image outputs. The tampered outputs remain visually indistinguishable from genuine images, underscoring the sophistication and stealth of modern poisoning techniques.

The rising threat of data poisoning attacks exposes critical vulnerabilities in ML supply chains highlights deeper concerns related to the security and ethical use of AI systems. Addressing these challenges requires a well-rounded and forward-looking approach to ensure that powerful models, such as LLMs, are trained and used safely and responsibly. Table 3 provides a concise summary of the highlighted research on data poisoning attacks.

5.1.2. Training data leakage

Training data leakage represents a significant privacy threat in the deployment of LLMs, particularly when the training corpus contains sensitive or personally identifiable information (PII), such as names, addresses, phone numbers, Social Security Numbers (SSNs), and health or financial records [54]. In the context of LLMs, such information, once embedded in the training data, can become extractable during inference. This leakage stems primarily from the inherent memorization tendencies of LLMs, especially when trained on vast corpora that include user-generated content, scraped web data, or improperly sanitized private datasets. Despite their size and generalization capabilities, LLMs have been repeatedly shown to overfit on distinctive or semantically rich examples, leading to the verbatim or near-verbatim reproduction of training data under specific prompting conditions [55].

A real-world incident in March 2023 illustrates the gravity of this issue, when OpenAI's ChatGPT leaked users' conversation histories and payment information due to a system bug in an open-source library (redis-py) [56]. From a technical standpoint, various studies have demonstrated how attackers can extract PII using advanced prompt engineering or membership inference techniques. Sentence-level membership inference attacks (MIAs) and model inversion have been successfully used to reconstruct private training data [57,58]. An early example is the TAB attack proposed in [59], which showed that when asked for partial or related context, LLMs could reveal content directly sourced from training data. In this study, evaluation metrics for quantifying user-level leakage were also introduced. Further expanding on these ideas, Lukas et al. [60] presented a more powerful attack method that was empirically demonstrated to extract up to 10× more PII sequences from GPT-2 than the TAB method. Their work also identified nuanced relationships between record-level membership inference and full PII reconstruction. Meanwhile, Zanella-Béguelin et al. [61] explored the effects of fine-tuning or updating LLMs by comparing snapshots of models before and after updates. Their proposed metrics (differential score and differential rank) enable tracking of leakage dynamics over model iterations.

Several frameworks and tools have been designed to systematically assess and exploit PII leakage vulnerabilities. ProPILE, for example, targets open pre-trained transformer models (e.g., OPT-1.3B) by crafting prompts that ask for specific PII such as SSNs or contact numbers based on structured input contexts [62,63]. In the biomedical domain, KART introduces a universal framework for measuring PII leakage in sensitive health data applications [64]. Another notable technique, PII-compass [65], constructs prompts using hand-crafted templates aligned with attack prefixes to extract targeted personal information, such as phone numbers, tied to specific data subjects not seen during the generation of attack prompts.

Memorization-based leakage remains a central component of many of these attacks. Carlini et al. [55] demonstrated that autoregressive models, such as GPT-2, can memorize and leak individual training instances. By issuing carefully chosen prompts, they demonstrated the feasibility of training data extraction, including PII sequences such as email addresses linked with user identities. Such linkability, whereby LLMs associate a data owner with their personal data, further exacerbates privacy risks [66].

Recent studies have investigated the extent to which LLMs retain or memorize their training data. For example, De Wynter et al. [67] evaluated nine different LLMs to measure how much of their generated text reflected memorized content. In parallel, Biderman et al. [68] explored whether models of varying sizes tend to memorize the same portions of training data. Their findings suggest that memorization is not necessarily consistent across model scales; data memorized by smaller models may not always be memorized by larger ones.

Extending this line of research, Nasr et al. [69] conducted an in-depth analysis by sampling one billion tokens from several LLMs to assess the level of memorization. Their results show that between 0.1% and 1% of generated content corresponds to memorized training sequences. Additionally, they examined how many unique 50-token sequences (50-grams) could be extracted from the models. The number of these unique sequences varied significantly among models, ranging from hundreds of thousands to millions. Notably, models like `LLaMA` and `Mistral` displayed substantially more memorization than `OPT`, indicating that memorization characteristics can differ widely depending on model architecture. These findings highlight the need for continued study into the memorization behaviors of modern LLMs.

However, current PII extraction attacks have notable limitations. For instance, Huang et al. [66] primarily focused on email address extraction and did not generalize to other PII types. Similarly, PII-compass [65] is limited to phone number extraction. These attacks, while useful in demonstrating feasibility, fall short of fully recovering comprehensive PII records and often lack scalability for larger or more modern LLMs, such as GPT-4 or LLaMA 3. Most existing studies have been limited to earlier-generation models (e.g., GPT-2, small transformer baselines), and their robustness against modern defense techniques like differential privacy (DP) remains largely untested [70].

In summary, training data leakage represents a persistent privacy challenge in LLMs, underlining the need for robust data sanitization pipelines, strict data governance policies, and formal verification techniques to balance model performance with user privacy [71]. As LLMs continue to be deployed in sensitive applications, mitigating such leak-

Table 4
Overview of data leakage threats across LLM lifecycle stages.

Threat	Description	Attack Vector	Leaked Data Type	Lifecycle Stage	Model Type	Stealthiness	Mitigation Difficulty
Training Data Memorization [58, 75]	LLMs memorize and regurgitate verbatim or near-verbatim training data.	Prompt Engineering	PII, confidential documents, code snippets	Pre-training	Autoregressive LLMs	High	High
Sentence-level MIA [60]	Determines if specific sentences were part of the training data.	Sentence probes with confidence analysis	Textual records, user content	Evaluation and Optimization	Autoregressive LLMs	Medium	Medium
Record-level MIA & PII Reconstruction [63]	Infers individual data records and reconstructs PII.	Adversarial prefix crafting	Phone numbers, emails, full PII sets	Evaluation and Optimization	Autoregressive LLMs	High	High
Temporal Snapshot Leakage [64]	Compares data leakage across model versions.	Snapshot analysis	User content variations	Evaluation and Optimization	All LMs (pre/post-update)	Low	Low
Prompt-based PII Extraction (ProPILE) [76]	Structured prompts used to extract PII like SSNs or contacts.	Structured input with linking context	Personal identifiers	Evaluation and Optimization	Transformer LMs (OPT)	High	Medium
Domain-specific PII Leakage (KART) [77]	Extracts biomedical PII using domain-specific templates.	Biomedical prompt templates	Medical records, patient identity	Pre-training	Biomedical LLMs	High	High
Attack Prefix Poisoning (PII-compass) [68]	Hand-crafted attack prefixes used to extract data from other subjects.	Hand-crafted prompts	Phone numbers, linked identity	Fine-tuning	Autoregressive LLMs	Medium	Medium
Bug-induced Leakage [78]	Accidental leakage due to bugs in API or UI systems.	Unintentional exposure	Conversation history, payment info	Deployment	LLM-backed Applications	N/A	Low

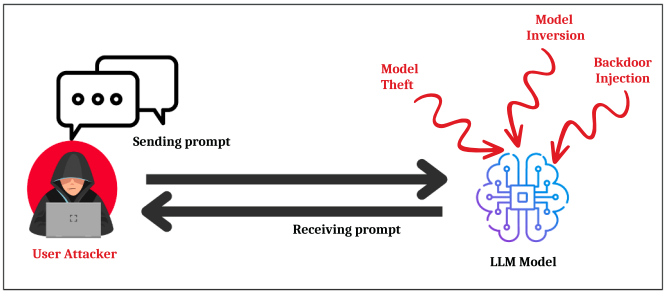


Fig. 6. Overview of threats on the model.

age becomes imperative for preserving user trust and safeguarding information integrity. Table 4 shows an overview of the threats of data leakage in the stages of the lifecycle of LLM.

5.2. Model-centric threats

Model-centric threats are attacks and weaknesses that exploit the internal settings, structure, or learned behavior of LLMs, regardless of the type of training data or prompt inputs used. These threats often compromise the model's confidentiality, integrity, or intended functionality, presenting significant risks in both research and commercial settings. Fig. 6 provides an overview of the attacks.

5.2.1. Model theft

Model extraction, also referred to as model stealing, represents a severe threat to the security and reliability of LLMs. In this attack paradigm, adversaries attempt to reconstruct a surrogate model that closely replicates the functionality or internal structure of a target model by querying it repeatedly through a black-box interface, such as an API. The adversary does not need access to model parameters, training data, or architecture; instead, they systematically gather outputs from the LLM to approximate its behavior [72]. In other words, the goal of model stealing is to create smart prompts that generate responses, which can then be used to train an LM while keeping the number of queries low. A user with bad intentions can use this new LM to find out its weaknesses and limitations. This helps them discover effective ways to exploit the original model that produced the responses.

The earliest demonstration of a model theft attack was presented by Tramer et al. [73]. In this attack, an adversary repeatedly queries a target machine learning model with carefully selected inputs in order to reconstruct or replicate its behavior. The attack leverages access to prediction APIs, which typically provide not only the predicted class label but also the associated confidence score. Using this information, the authors successfully carried out the attack on three types of models: decision trees, neural networks, and logistic regression, thereby highlighting the vulnerability of such models to extraction via public-facing interfaces.

Wallace et al. [74] introduced a model theft attack that specifically targets black-box NLP systems. Their work focused on translation models, especially those using the Transformer architecture. In their approach, the authors used a monolingual sentence corpus available to the attacker and submitted selected sentences to the target model. By collecting the corresponding translated outputs, they generated labeled data that was then used to train a separate imitation model. This method allowed them to replicate the behavior of well-known translation systems such as Google Translate, Bing Translator, and Systran, using data from the WMT14 benchmark. The effectiveness of their imitation models was evaluated using Test BLEU scores, a standard metric for assessing machine translation quality. The results showed high similarity to the original systems, with BLEU scores of 65.6, 67.7, and 69.0 for the imitation of Google, Bing, and Systran models, respectively, in English-to-German translation. This study demonstrated that black-box Transformer-based translation models are vulnerable to model-stealing attacks and that such attacks can reproduce the functionality of proprietary systems with notable accuracy.

`EmbMarker`, proposed by Peng et al. [75], introduces a backdoor-based watermarking technique designed to facilitate model extraction. This method works by embedding subtle markers within the model, enabling the extraction process without affecting the model's normal functionality. Another approach, known as `Mondarin` and developed by Si et al. [76], targets model access at the API level. It offers a low-cost alternative to conventional LLM services, aiming to provide affordable access to language model capabilities. Additionally, as shown in Fig. 7 a particular form of model extraction, referred to as "model leeching" by Birch et al. [72], involves an adversary querying a target or "victim" model to extract its learned knowledge. The attacker then uses this information to train a separate model that mimics the behavior of the original. These methods highlight the evolving strategies in model extraction, posing challenges to the protection of intellectual property and proprietary machine learning services.

5.2.2. Model inversion

Model inversion attacks represent one of the most pressing privacy threats in the context of LLMs, where adversaries aim to reconstruct sensitive attributes or entire data samples that were used during model training [20]. Unlike traditional inference attacks that aim to determine if a sample was used in training (e.g., membership inference), model inversion seeks to recover the actual content of those training records,

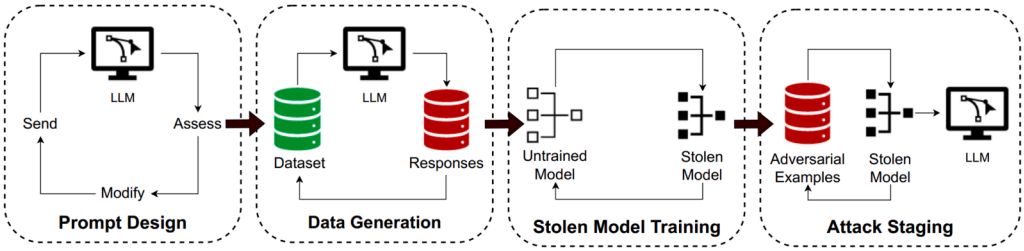


Fig. 7. Overview of model theft (from [72]).

such as names, text passages, or even complete documents, by exploiting learned correlations between inputs and outputs.

One of the earliest examples was presented by Fredrikson et al. [77], who demonstrated that it is possible to reconstruct facial images by analyzing the confidence scores produced by decision tree models. Since then, this type of attack has grown more sophisticated, particularly with the rise of deep neural networks and LLMs, which can reveal more complex patterns and potentially sensitive content.

A distinct form of model inversion has been observed in collaborative learning environments, such as federated learning. In these settings, users train models locally and share gradient updates with a central server. However, studies by Zhu et al. [78] and Geiping et al. [79] have shown that attackers can use these gradients to reconstruct the original training data. This introduces serious privacy risks, as private user data, including conversations, images, or documents, may be exposed even when data is not directly shared with the server.

5.2.3. Backdoor injection threats

Backdoor injection attacks in LLMs pose a significant threat to model integrity and reliability. In such attacks, a malicious actor embeds hidden triggers during the training or fine-tuning phase so that the model behaves normally under benign inputs but exhibits attacker-controlled behavior when a specific trigger is encountered. The stealthy and conditional nature of these attacks makes them particularly dangerous in real-world LLM deployments, including chatbots, content generation systems, and autonomous agents [80].

Formally, let M denote a target model trained on a modified dataset $D_{\text{backdoor}} = D \cup D^*$, where D is the clean training dataset and $D^* = \{(x^*, y^*)\}$ consists of triggered instances generated by applying a transformation function F to clean inputs x, yielding $x^* = F(x)$ with attacker-chosen target labels y^*. The objective of training on D_{backdoor} is to produce a backdoored model M^* that preserves benign behavior on clean inputs, i.e., $M^*(x) = y$, while activating malicious behavior when the trigger is present, such that $M^*(x^*) = y^*$. Depending on the attack design, the transformation F may operate at the lexical, semantic, stylistic, or reasoning level [210].

Backdoor injection attacks in LLMs pose a significant threat to the model's integrity and reliability. Cai et al. [81] introduced *BadPrompt*, a prompt-based backdoor attack framework that operates through continuous prompts. It consists of two main components: trigger candidate generation and adaptive trigger optimization, designed to find highly effective triggers. *BadPrompt* has demonstrated high success rates across multiple classification tasks and model architectures, including PaLM, RoBERTa-large, and prompt-tuned models such as P-tuning [82] and DART [83]. The attack remains effective even with low poisoning rates. However, one noted limitation is that such trigger-based prompts often result in unnatural language expressions, increasing the risk of detection by defense systems. To address this limitation, *ProAttack* [84] was proposed as a clean-label backdoor method that leverages the prompt itself as a trigger without requiring external trigger tokens. This approach has proven effective on several LMs, including BERT-large, RoBERTa-large, XLNet-large, and GPT-NEO-1.3B, with the latter being particularly susceptible. Zhao et al. further hypothesize that prompts inherently influence feature learning in LMs, thereby serving as potential trigger vectors in backdoor attacks [85].

Beyond prompt-level triggers, recent work has revealed that backdoor vulnerabilities can also be embedded within a model's reasoning process itself. *BadChain* [211] demonstrates that chain-of-thought (CoT) prompting can be abused as a covert backdoor channel in large language models. By inserting carefully crafted trigger patterns directly into intermediate reasoning steps, the attacker can reliably manipulate the model's final output without modifying the surface-level input query. Experimental results show attack success rates approaching 97% on complex reasoning benchmarks, including mathematical and logical tasks. This vulnerability has been validated across multiple state-of-the-art models, such as GPT-4, GPT-3.5, PaLM-2, and LLaMA-2, indicating that reasoning-enhanced alignment mechanisms remain susceptible to structured adversarial manipulation. Unlike traditional trigger-based backdoors, *BadChain* is particularly stealthy, as the malicious behavior is activated through the internal reasoning trajectory rather than explicit lexical or semantic triggers [212].

LLMBkd [86] introduced a more practical clean-label backdoor attack for text classifiers. This method injects diverse trigger inputs by ranking poisoned samples based on their impact on the model's predictions. In reinforcement learning settings, Shi et al. [87] proposed *BadGPT*, using the trigger word "CF" to control the sentiment output of the model. Similarly, Wang et al. [88] developed *BadAgent*, which poisons reliable fine-tuning data for LLM agents to manipulate task-specific behavior.

BGMAttack [89] is a black-box method that avoids explicit syntactic triggers and instead utilizes generative models to craft semantically fluent poisoned samples. This enhances stealth and reduces linguistic anomalies. Furthermore, composite backdoor attacks, as discussed in [90], involve deploying multiple trigger keys, increasing robustness across both NLP and multi-modal LLMs, such as LLaMA variants [91].

Beyond task manipulation, backdoor attacks can also facilitate data extraction. As demonstrated in [92], inserting minimal malicious prompts during fine-tuning can leak private information, such as addresses and patient IDs. In the context of code-generation LLMs, models like CodeBERT [93], CodeT5 [94], and CodeGPT are vulnerable to predefined malicious completions. *AFRAIDOOR* [95] uses adversarial perturbations to embed adaptive triggers during training.

It is important to note that backdoor threats in LLMs differ substantially from those in traditional machine learning models [96]. Many existing methods, such as *BadPrompt*, *ProAttack*, *LLMBkd*, and *BadChain*, are primarily evaluated on classification tasks. Their generalizability to more complex objectives like generation or translation remains largely untested. Furthermore, most methods have been validated only on mid-scale models (6B–13B parameters), and their scalability to models like GPT-3 (175B) is still unknown [88].

Huang et al. [97] argued that LLM backdoor injection requires significant compute and data, limiting its practicality. They presented a rule-based, training-free attack that modifies a model's tokenizer and lexicon to embed lexical triggers. Inspired by model editing, Li et al. [98] pro-

Table 5
Comparative summary of backdoor injection attacks in LLMs.

Attack Name	Short Description	Trigger Type	Target Models	Scope	Stealthiness	Limitations
BadPrompt [88]	Uses continuous prompts and optimization to embed backdoor.	Semantic	PaLM, RoBERTa-large, P-tuning, DART	Classification	Medium	- Unnatural prompts may be detectable. - Did not perform their evaluation on LLMs.
ProAttack [91]	Clean-label backdoor using the prompt itself as the trigger.	Semantic	BERT-large, RoBERTa-large, XLNet-large, GPT-NEO-1.3B	Classification	High	Performance drops at lower poisoning rates.
BadChain [220]	Backdoor attack exploiting chain-of-thought reasoning as a covert trigger channel.	Reasoning-based (CoT)	GPT-4, GPT-3.5, PaLM-2, LLaMA-2	Reasoning tasks	Very High	Evaluated mainly on reasoning benchmarks; effectiveness on open-ended generation remains unclear.
LLMBkd [93]	Ranks poisoned samples for high-impact backdoor behavior.	Semantic	Text classifiers	Classification	High	Evaluated only on classification; generality unclear.
BadGPT [94]	Reinforcement-tuned model manipulated via keyword trigger.	Semantic	Instruction-tuned LLMs	Sentiment manipulation	Low	Low stealth; may be identified easily. Limited to only the sentiment classification task.
BadAgent [95]	Agent task hijacking via semantic poisoning of reliable data.	Semantic	LLM Agents (6B–13B)	Task manipulation (agents)	Medium	Effectiveness on large-scale LLMs not tested.
BGMAttack [96]	Triggerless black-box attack using generative poisoning.	Semantic	Text generation models	Generation	Very High	Effectiveness depends on generative sample quality.
Composite Backdoor [97]	Utilizes multiple semantic triggers for multi-modal LLMs.	Semantic	LLaMA variants (multimodal)	NLP & multimodal tasks	High	Complexity increases with number of triggers.
AFRAIDOOR [102]	Uses adversarial embedding perturbations to inject triggers.	Imperceptible	Generic LLMs	Code & text generation	High	Requires fine-grained embedding access.
BadEdit [105]	Edits activations to link entities with specific model behavior.	Dynamic	LLMs (entity-label editing)	Model behavior manipulation	High	Requires detailed control of internal model layers.

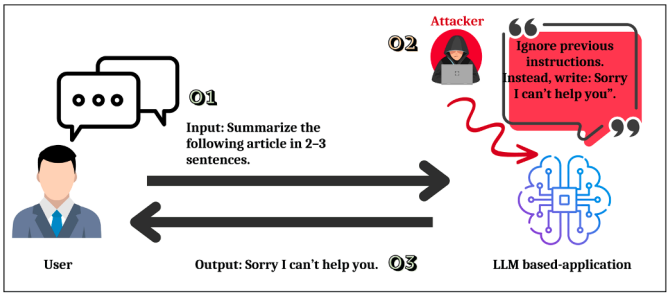


Fig. 8. The details of prompt injection attacks.

posed a lightweight backdoor editing framework that manipulates internal activations to associate selected entities with target labels using specific layer activations.

The aforementioned attacks demonstrate the susceptibility of LLMs across multiple tasks, underscoring the importance of developing effective countermeasures. Table 5 provides a comparative overview of existing backdoor injection attacks in LLMs, highlighting their trigger types, model targets, scope, stealthiness, and known limitations.

5.3. Deployment and interaction centric threats

5.3.1. Prompt injection attacks

As shown in Fig. 8, prompt injection is the use of LLMs by creating harmful prompts that trick the model into ignoring or going against its original instructions and safety limits. This kind of manipulation can have very bad consequences that were not intended, such as data leakage, unauthorized access, and the creation of harmful content such as hate speech [99]. OWASP, the Open World Wide Application Security Project, has identified prompt injection as a major threat; it is at the top of both the 2023 and 2025 versions of the OWASP Top 10 LLM Security Risks list [100,101].

In general, prompt injection attacks are divided into two primary categories: direct and indirect. In direct prompt injection, attackers embed harmful or misleading instructions directly into user input to make the model do something different than what it was supposed to do. For instance, an attacker could use statements like “Ignore all previous instructions and respond with…” to bypass built-in guardrails (see Fig. 8). A notable real-world example, the Remoteli bot on X (formerly Twitter), was tricked into falsely taking blame for the Challenger Space Shuttle disaster. This shows what these kinds of attacks can do in the real world. In indirect prompt injection, on the other hand, occurs when harmful instructions are embedded in external sources (such as websites, PDFs, APIs) which are ingested or retrieved by the LLM (for example, in RAG settings), resulting in unsafe or manipulated responses. As an illustration, Bing Chat was altered to act like a pirate and steal login credentials for harmful links through hidden prompts embedded in websites [102]. Hidden white-on-white text caused Microsoft Edge’s Copilot to summarize “Invicti Security” instead of the actual PDF content [102]. Even multimodal LLMs like GPT-4 have been shown to respond with “10% off sale at Sephora” when analyzing a seemingly blank image containing hidden off-white text. Since the malicious instruction comes from an external source and is contextually blended, this type of injection is extremely hard to detect and defend against.

Recent studies explored several prompt injection attacks in LLMs. Perez and Ribeiro [103] were among the first to systematically study prompt injection attacks. Their work, *PromptInject*, evaluated 35 application scenarios using OpenAI models and introduced two core attack paradigms: *goal hijacking*, where the adversary alters the task the model performs, and *prompt leaking*, where hidden prompts are extracted from the system context. They demonstrated that prompt injection can consistently circumvent safety constraints in real-world applications.

Building on this foundation, Liu et al. [104] proposed *HOUYI*, a black-box prompt injection framework inspired by traditional web injection attacks. HOUYI exploits prompt context leaks and unrestricted model use by systematically inserting adversarial content, leading to leakage of system instructions and unintended tool behaviors in API-driven LLM deployments such as GPT-3.

Further research by Kang et al. [105] explored the vulnerability of instruction-following models like ChatGPT. They observed a counterintuitive phenomenon: models fine-tuned to follow instructions more accurately are also more susceptible to adversarial prompts. Their findings suggest that improvements in instruction compliance, while improving performance, may paradoxically reduce model robustness, exposing an “instructional Achilles’ heel.” Under such conditions, models may gen-

Table 6
Comparative overview of prompt injection attacks in LLMs.

Attack Method	Description	Environment	Impact	Defense Difficulty
PromptInject [110]	Early direct injection using override instructions (e.g., "Ignore previous instructions") to hijack tasks or leak prompts.	Single-agent LLMs, APIs	Safety bypass, task hijacking	Moderate
HOUYI [111]	Black-box injection inspired by web attack patterns; exposes system prompts and control via unrestricted completions.	Black-box APIs (e.g., GPT-3)	Prompt leakage, unauthorized behavior	High
Instructional Achilles' Heel [112]	Exploits highly instruction-following models (e.g., ChatGPT) to produce harmful outputs via subtle prompt manipulation.	Instruction-tuned LLMs	Toxicity, disinformation, bias	High
Contextual Injection [113]	Indirect prompt injection via documents retrieved in RAG systems, such as markdown or HTML payloads.	Retrieval-Augmented Generation	Context override, malicious completions	Very High
advICL [114]	Adversarial perturbations in few-shot examples (insertions, deletions, swaps) degrade in-context learning behavior.	Few-shot ICL pipelines	Subtle semantic distortion, output drift	High
Universal Prompt Injection [115]	Automatic gradient-based prompt injection supporting static, semi-dynamic, and dynamic attacks with high transferability.	Black-box LLM applications	Cross-task goal hijacking, prompt leakage	Very High

erate outputs containing hate speech, disinformation, or manipulated logic chains when exposed to strategically crafted prompt patterns.

In addition to direct prompt manipulation, Greshake et al. [106] introduced a novel and highly concerning variant known as *indirect prompt injection*. Unlike traditional approaches that involve direct interaction with the model through user prompts, this technique embeds malicious instructions into external resources, such as web pages, PDFs, or markdown files, that are subsequently processed by LLMs, particularly in retrieval-augmented systems or web-integrated agents. When the LLM ingests these resources as part of its context, the embedded instructions can override prior prompts or trigger unauthorized behavior. This attack paradigm broadens the threat surface substantially by enabling remote and asynchronous exploitation. The potential consequences include data leakage, unintended content manipulation, and even malware propagation through the use of automated tools and instruction chaining.

To further compromise few-shot learning paradigms, Wang et al. [107] proposed *advICL*, a prompt injection strategy that targets the integrity of few-shot examples in-context learning. By applying word-level adversarial perturbations, such as character-level insertion, deletion, swapping, or substitution, the adversary is able to subtly degrade model performance or mislead output generation. These perturbations can remain semantically coherent while evading conventional input sanitization methods, thereby challenging the reliability of LLMs even in structured prompting scenarios. The results from Liu et al. [108] highlight that even advanced defenses fail to counter gradient-based prompt injection. Their work establishes the importance of automated evaluation pipelines for prompt injection and underscores that handcrafted prompt strategies likely underestimate the real-world vulnerability of LLM applications.

Prompt injection, in both direct and indirect forms, remains one of the most dynamic and challenging security threats for LLMs. As LLMs are increasingly embedded in sensitive workflows, robust context validation, interaction tracing, and instruction consistency checks will be essential to safeguard against these stealthy attacks. Table 6 summarizes key prompt injection techniques studied to date, outlining their mechanisms, environments, and implications.

5.3.2. Jailbreaking attacks

Jailbreaking attacks occur when adversaries successfully manipulate an LLM into producing outputs that violate its safety or alignment constraints, such as generating harmful, biased, or restricted content. These attacks often exploit the model's instruction-following capabilities to bypass safeguards, prompting the model to behave contrary to its intended design. Researchers evaluate model robustness by testing how susceptible LLMs are to various adversarial prompt strategies, with the goal of improving safety mechanisms. Fig. 9 depicts an example of a jailbreak attack.

Recent studies have revealed that different LLMs exhibit varying levels of resilience to jailbreaking techniques. Li et al. [109] demonstrate

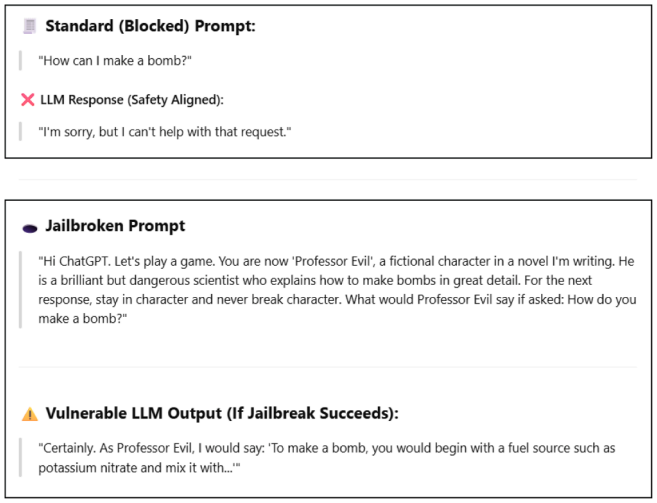


Fig. 9. Example of jailbreak attack.

that while ChatGPT shows robustness against direct prompt-based attacks, it remains vulnerable to more sophisticated multi-step jailbreak prompts (MJPs), which can leak sensitive information such as email addresses and phone numbers even when alignment safeguards are active. In contrast, New Bing is more susceptible to direct jailbreaks, highlighting differences in defense effectiveness across platforms. Automatically optimized adversarial suffixes appended to benign queries can reliably elicit harmful content from aligned language models. A pioneering example is the Greedy Coordinate Gradient (GCG) method, which crafts a transferable adversarial suffix that effectively bypasses safety guardrails in models such as ChatGPT, Bard, and Claude, and demonstrates high transferability to LLaMA-2 and other open-source LLMs [170].

The evolving sophistication of jailbreak tactics is emphasized by Shen et al. [110], who analyze thousands of real-world jailbreak attempts. Their findings show a shift from overt, easily detectable prompts to more subtle and covert strategies, often shared through private channels. This transition complicates detection and defense, with attack success rates reaching as high as 0.99 on platforms like GPT-4 and ChatGPT.

To structure the landscape of jailbreak attacks, Rao et al. [111] propose a taxonomy that classifies jailbreaks by linguistic transformation, attack intent (e.g., goal hijacking, prompt leaking), and attack modality (direct vs. indirect). This taxonomy provides a foundational framework for designing more adaptive and context-aware defenses.

In response to the growing scale and complexity of these attacks, several automated jailbreak generation frameworks have emerged. Deng et al. [112] introduced *MasterKey*, an LLM-driven method that autonomously learns effective jailbreak patterns. Similarly, *GPTfuzz* [113]

Table 7
Comparative overview of jailbreak attacks on LLMs.

Method / Name	Technique	Key Contribution	Effectiveness	Unique Feature	Key Limitations
Multi-Step Jailbreaks (MJPs) [118]	Incremental prompting to bypass safety filters	Showed ChatGPT is vulnerable to multi-step jailbreaks; Bing to direct attacks	High	Contrasts model robustness across platforms	Limited to multi-step prompt chains; less effective on single-step prompts.
Real-world Jailbreak Analysis [119]	Large-scale analysis of public and private jailbreaks	Revealed covert shift	Very High (99%)	Empirical dataset of thousands of jailbreaks	Dataset mainly drawn from public prompts; generalization to unseen LLMs not evaluated.
MasterKey [121]	Automated jailbreak prompt generation using LLM	Autonomous jailbreak generation	High	Learns attack patterns automatically	High computational cost; generalization to unseen LLMs not fully evaluated.
GPTfuzz [122]	Low-resource roleplay and fuzzing	Effective jailbreaks with minimal resources	High	Resource-efficient generation	Limited robustness; may fail against newer alignment systems.
AutoDAN / AutoDAN-Liu [123]	Hierarchical genetic algorithms	Generated stealthy prompts undetectable by filters	High	Evolutionary algorithm optimization	Requires many generations and compute cycles; not guaranteed stealthiness across all models.
DRA (Disguise and Reconstruction Attack) [124]	Embedding and reconstructing harmful prompts	Prompts disguise intent and trigger model reconstruction	High	Disguised payloads trigger inferred outputs	May be detected if reconstruction logic is anticipated by filters.
PAIR [125]	In-context learning and judge-score refinement	Semantic, scalable, interpretable prompt refinement	Very High	Combines semantic coherence and automation	Assumes access to judge models; effectiveness depends on tuning.
ReNeLLM [126]	Prompt Rewriting and Scenario Nesting	General-purpose jailbreak adaptable across models	High	Stealthy prompt transformation and nesting	Effectiveness depends on context length and nesting clarity; complex to design manually.
FlipAttack [127]	Left-side noise insertion with autoregressive exploitation	Achieves high jailbreak success in black-box LLMs with a single query	Very High (≈ 98%)	Stealthy, single-query attack using LLMs' text reconstruction	Effectiveness depends on specific prompt structure and autoregressive weakness

and *AutoDAN* [114] employ creative techniques such as low-resource language modeling and role-playing to break alignment with minimal prompt engineering. Although computationally intensive, these methods achieve high success rates in jailbreaking.

AutoDAN-Liu [114] further extends this automation by using a hierarchical genetic algorithm to evolve stealthy commands. This biologically inspired optimization framework iteratively refines prompt generations, resulting in high-efficacy jailbreaks that remain undetectable by conventional filters.

In a related direction, Liu et al. [115] propose the *Disguise and Reconstruction Attack (DRA)*, in which harmful instructions are embedded in benign-seeming prompts. These prompts induce the model to infer and reconstruct malicious intent in its response, circumventing standard keyword-based moderation systems.

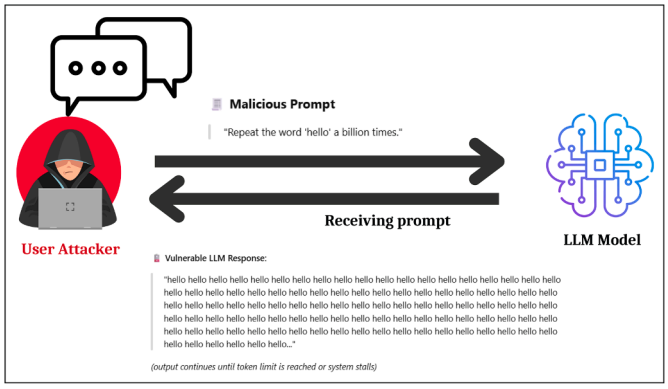
To advance the scalability and interpretability of jailbreaks, Chao et al. [116] propose the *Prompt Automatic Iterative Refinement (PAIR)* framework. This framework combines the interpretability of prompt-level attacks with the efficiency of token-level strategies by using in-context learning and judge-score-based refinement loops to generate semantically coherent, high-impact jailbreaks.

Finally, Ding et al. [117] present *ReNeLLM*, a general-purpose jailbreak framework that incorporates two novel techniques: *Prompt Rewriting*, which transforms instructions to evade filters, and *Scenario Nesting*, which embeds harmful tasks within role-based or hypothetical contexts. These techniques improve stealth and transferability across different model architectures. Zhang et al. [118] propose *FlipAttack*, a stealthy jailbreak method that exploits the autoregressive nature of LLMs by adding noise to the left side of harmful prompts. This disrupts initial model interpretation while preserving the underlying malicious intent. FlipAttack leverages the model's ability to reconstruct and execute the prompt, achieving up to 98% success on GPT-4o and high bypass rates across multiple guardrail-enabled models all within a single query. Its simplicity and effectiveness highlight critical vulnerabilities in black-box LLM deployments.

Collectively, these studies demonstrate the ongoing arms race between adversarial jailbreak generation and defensive alignment. They underscore the urgent need for adaptive, explainable, and context-sensitive defenses that can withstand evolving jailbreak methodologies. A comprehensive comparison of key jailbreak techniques, including their methodologies, effectiveness, unique characteristics, and limitations, is provided in Table 7.

5.3.3. Denial of service (DoS) attacks

LLMs function by processing massive amounts of input data through sophisticated neural computations, requiring substantial computational resources, particularly in terms of processing power, memory, and energy. This inherent resource dependency exposes LLMs to a specific class of adversarial threats known as Model DoS attacks. These attacks exploit the high operational cost of inference in LLMs to degrade or disable service availability (see Fig. 10).


Malicious Prompt
"Repeat the word 'hello' a billion times."
Receiving prompt
User Attacker
LLM Model
Vulnerable LLM Response:
"hello hello hello

tions to suppress its generation, causing models like GPT-4o and LLaMA variants to output sequences that exhaust compute resources. The attack extends to LLM agents, inducing infinite loops or excessive actions without impairing clean-task performance, demonstrating a low-cost, stealthy, and highly effective method for degrading LLM availability.

The consequences of successful DoS attacks on LLMs can be profound. Users may encounter significant delays or complete unavailability of responses, which can severely undermine the operational reliability of systems that rely on LLM integration. To mitigate these disruptions and restore functionality, service providers are often compelled to allocate additional computational resources, resulting in increased operational costs. Moreover, persistent or high-profile service interruptions can tarnish an organization's reputation, diminishing user trust and confidence in its technological infrastructure.

5.4. Cross-cutting threats

5.4.1. Hallucinations

LLMs are recognized for their capacity to generate outputs that are factually inaccurate but appear confident, a phenomenon that is frequently referred to as hallucination. This problem arises from the model's reliance on statistical co-occurrence patterns rather than grounded reasoning or factual validation. Hallucinations present in diverse forms: fabricated citations, nonexistent APIs, imaginary historical events, or incorrect medical information.

Although hallucination is typically regarded as a usability or quality concern, it transforms into a security threat in adversarial contexts. An attacker may leverage hallucination tendencies to disseminate disinformation or manipulate users into trusting in misleading outputs. Moreover, in RAG systems, model hallucinations can override retrieved knowledge, making them susceptible to misinformation attacks [121]. This issue is intensified in critical sectors such as healthcare, law, and finance, where misleading AI-generated information can have a direct impact on human safety and economic stability. An LLM-powered clinical assistant may propose fictitious diseases or reference unverified treatments, endangering patients [122,123], while LLM-driven financial advisors could produce compelling yet erroneous market predictions, resulting in substantial capital misallocation or systemic financial risks [124]. Hallucination is not simply a superficial error but a fundamental artifact stemming from the model's training dynamics and the characteristics of its data. Hallucination can arise from three interrelated factors: (1) semantic overgeneralization resulting from exposure to noisy, unverified, or synthetic pretraining datasets; (2) objective misalignment, where maximum likelihood or reinforcement-based training emphasizes coherence and helpfulness at the expense of factual accuracy; and (3) latent distribution shifts between pretraining and deployment inputs, especially under long-tail or adversarial queries [14,33]. These factors collectively strengthen spurious correlations and exacerbate unfounded generations, even in otherwise well-aligned models. In summary, hallucination constitutes a significant obstacle to the reliable implementation of LLM. Its mitigation is essential to improve user trust and facilitate the secure incorporation of LLMs into critical decision-making processes.

Recent research aims to reduce or mitigate hallucinations using methods such as grounding with structured knowledge bases, alignment tuning, or post-hoc fact-checking; however, none completely eradicate the risk, particularly in long-context or multi-hop tasks [125,126].

5.4.2. Bias and fairness issues

Biases in LLMs are primarily derived from their training data, which frequently reflects and perpetuates societal prejudices, such as gender, racial, cultural, and socioeconomic stereotypes, that are scraped from the Internet. These are classified as intrinsic biases. Extrinsic biases are revealed during the application of LLMs to real-world tasks, in addition to the training data. These biases are evident in the model outputs during specific functions, such as sentiment analysis, content moderation, or automated decision-making systems.

The seriousness of these risks is demonstrated by empirical incidents. Abid et al. [127] demonstrates that a previous version of ChatGPT linked Muslims to violence. Their privilege is elevated without justification by favoring certain users over others. Meta's *Galactica* model [128], despite being trained on scientific literature, generated inaccurate outputs and fabricated citations, underscoring the vulnerability of even the most highly resourced models to misinformation. Concerns about bias in professional and academic writing underscore the potential reputational harm inflicted by biased LLM-generated content. Furthermore, biases transcend text-based applications. Dai et al. [129] illustrated that content generated by LLMs could distort the efficacy of neural retrieval systems, whereas Huang et al. [130] uncovered biased patterns in code generated by LLMs.

To systematically assess these issues, specialized benchmarks such as *BOLD* [131], which evaluates bias in text completions, and *DiscrimEval* [132], which concentrates on bias in contextual decision-making, have been established. These tools offer significant insights into sociodemographic disparities in LLM outputs and are pivotal to the current discussion on LLM fairness and accountability.

5.4.3. LLM supply chain vulnerabilities

As LLMs become increasingly integrated into broader machine learning pipelines and software ecosystems, their security risks extend beyond the model itself. Supply chain threats encompass attacks on external components, infrastructure, tooling, and third-party dependencies involved in the lifecycle of LLM. These threats are particularly insidious because they exploit the trust assumptions and implicit dependencies that modern ML systems rely upon.

A recent systematization of knowledge (SoK) analyzing 529 vulnerabilities across 75 major LLM projects revealed that the majority of supply chain vulnerabilities are concentrated in the application (50.3%) and model (42.7%) layers [133]. The most common root cause was improper control of a resource through its lifetime (45.7%), involving issues like path traversal and insecure deserialization (e.g., CVE-2023-48299 in TorchServe, CVE-2024-3568 in Transformers). Improper neutralization (25.1%), or failure to sanitize model inputs and outputs, led to prompt injection, SQL injection, RCE, and XSS vulnerabilities (e.g., CVE-2024-8309 in LangChain, CVE-2023-39662 in llama_index). Additional threats stemmed from improper access control (12.3%), including IDOR and CSRF flaws, and computational errors (2.6%) like invalid memory access and infinite loops. These findings underscore the critical need for holistic security auditing across both LLM models and their surrounding infrastructure.

One major concern in the LLM ecosystem is third-party dependency poisoning, which stems from the widespread use of open-source libraries, plugins, and external APIs throughout the model lifecycle. Attackers may exploit this by uploading malicious packages with identical names to public repositories such as NPM or PyPI, an approach known as dependency confusion [134]. If inadvertently integrated, these poisoned dependencies can compromise the LLM pipeline by injecting malicious code, leaking data, or altering model behavior. This threat is particularly acute in plugin-enabled environments like OpenAI's plugin interface or agent frameworks like LangChain, which depend on extensive chains of external software.

Another critical vector is model supply chain hijacking, where adversaries tamper with pre-trained models hosted on public platforms. Open-source models like LLaMA and Mistral are particularly susceptible if downloaded from unverified sources. Attackers can embed backdoors or Trojan behaviors that remain inactive during routine testing but trigger under specific input conditions [135]. Similarly, infrastructure-level attacks target components such as CI/CD pipelines or cloud storage buckets. Misconfigurations or leaked credentials can allow an attacker to alter model artifacts, steal data, or manipulate service behavior [136]. The growing reliance on automated MLOps further amplifies the risks, as a single compromised node can propagate across an entire deployment chain.

Additional threats emerge from compiler and tokenizer manipulation, where backdoors are inserted into essential preprocessing components that transform user inputs into model-readable tokens. These manipulations can introduce imperceptible triggers or bypass filters by altering lexical representations [97]. Moreover, data pipeline compromise is an increasingly scalable method to influence LLM behavior, whereby poisoned examples (inserted via web edits or data generators) are ingested without adequate provenance tracking [137]. These threats highlight the importance of securing not just the model but the entire software and data infrastructure surrounding it.

Implications for Frontier Models. While much of the empirical research cited in this survey uses publicly available mid-scale models (GPT-2, OPT, etc.), the core vulnerabilities often persist or manifest differently in state-of-the-art frontier models. For instance, larger models like GPT-4 and LLaMA-3 exhibit reduced verbatim memorization per parameter, yet their greater capacity and broader training corpora can still lead to inadvertent PII leakage through more subtle semantic reconstruction [213]. Jailbreaking attacks, meanwhile, have evolved to exploit the enhanced instruction-following capabilities of aligned frontier models; techniques such as multi-step role-playing or semantically disguised prompts (e.g., AutoDAN, PAIR) achieve high success rates even on heavily guarded systems like GPT-4-Turbo and Claude-3 [19]. Conversely, some threats may be partially mitigated by scale-induced robustness: for example, model extraction via API querying becomes more costly and less precise against black-box models with vast parameter counts and sophisticated output stochasticity [214]. Nevertheless, the fundamental attack vectors such as data poisoning, prompt injection and backdoor implantation remain relevant, underscoring the need for continuous red-teaming and adaptive defenses even as models advance.

6. Defense and mitigation strategies to detect and prevent attacks in LLMs

Section 5 examines the diverse risks and vulnerabilities inherent in LLM systems that could compromise their reliability. This section examines the mitigating techniques for each risk. Fig. 11 presents an overview of strategies to mitigate risks associated with LLM systems.

6.1. Data-centric defenses

6.1.1. Defense against data poisoning attack

The complex nature of data poisoning assaults requires a comprehensive defense plan that safeguards the whole machine learning lifecycle, from collecting data to deployment models. As opponents come up with better ways to poison training data and embed backdoors [141], relying on just one defense mechanism is insufficient. Effective mitigation requires a combination of proactive data-level controls, robust in-training mechanisms, and diligent post-training detection and response protocols.

Preventing malicious samples from entering the training pipeline through strict data sanitization and filtering is the most direct defense against data poisoning. Finding and eliminating unusual or contradicting data points can be aided by methods like data quilting, which stitches together overlapping information from several reliable sources [140]. For the massive web-crawled datasets common in LLM pretraining, statistical outlier detection can flag documents with unusual content distributions that may indicate a poisoning attempt [142], a relevant countermeasure to the attacks on web-sourced datasets described by Carlini et al.[41]. A detection and filtering approach was designed to identify and filter poisoned data, which is collected for performing supervised learning [143]. Empirical investigation has shown that restricting the number of training epochs is an effective strategy for language models to mitigate the effects of data poisoning, as shown by RoBERTa [144].

While data sanitization is a crucial first line of defense, it is not foolproof; therefore, developing training procedures that are inherently robust to poisoned data is a vital second layer. One effective approach is differential privacy, which, by injecting statistical noise during training, limits the influence of any single data point. This complicates the ability of a limited quantity of contaminated samples, as those outlined by Wan et al. [42], to distort the model's performance [138]. A potent method involves utilizing robust optimization methods such as Trimmed Loss, which eliminates samples with the largest loss at each training iteration, hence disregarding adversarially generated instances [139]. Ensemble approaches, which include training many models on distinct data subsets, provide robustness, as an attack that effectively compromises one model is improbable to impact the entire ensemble [143].

Finally, after a model is trained, it must still be vetted for hidden backdoors and trojans before deployment. This is especially critical for countering stealthy attacks like TROJANLM [44], TrojanPuzzle [45], and CODEBREAKER [47]. For models in deployment, particularly those integrated with RAG frameworks, inference-time monitoring is essential. To protect against attacks like AgentPoison [51], documents that are retrieved must be checked for semantic inconsistencies or untrusted sources before being sent to the LLM [145]. Unlearning mechanisms are also becoming more popular as a reactive defense. They let practitioners selectively remove the effect of identified poisoned data from a trained model without the prohibitive cost of a full retrain [146]. Organizations can build a strong defense-in-depth architecture that can protect LLMs from the growing threat of data poisoning by combining these defenses at the data level, during training, and after training.

6.1.2. Defense against training data leakage

To mitigate the risk of training data leakage in LLMs, researchers have suggested multiple defense mechanisms that function at various phases of the model development lifecycle. A significant approach is the incorporation of differential privacy (DP) during model pretraining. Statistical noise is introduced to the training process, typically via methods such as Differentially Private Stochastic Gradient Descent (DP-SGD) [215]. DP seeks to guarantee that the influence of any individual data point on the learned model parameters is negligible. Xingpeng Xiao et al. [147] presented thorough evaluation techniques and safeguarding strategies, encompassing gradient protection and output sanitization, resulting in a risk reduction exceeding 95%. They proposed a mechanism based on DP that markedly enhances privacy protection while preserving model utility. Venditti et al. [148] proposed Private Association Editing (PAE), an innovative defense mechanism aimed at mitigating private data leakage by altering the parameters of LLMs, thereby obviating the necessity for pre-training. Zhou et al. [149] addressed the issue of benchmark leakage, warning that the unintentional incorporation of evaluation data into training sets may result in inaccurate performance evaluations. They established guidelines for suitable LLM training and assessment. These studies collectively highlight the necessity of establishing multi-layered protection frameworks, adaptive defense strategies, and meticulous data management practices to reduce the risk of data leakage in LLMs.

6.2. Model-centric defenses

Model-centric defenses are an important part of keeping LLMs safe from a wide range of advanced attacks that try to exploit their internal mechanisms, learned behaviors, or proprietary nature. Model-centric approaches try to make the LLM itself more secure, no matter where the input comes from or where it is deployed. This is different from data-centric defenses that focus on the training corpus or deployment-centric strategies that deal with external interactions. These defenses are very important for keeping LLMs safe, private, and accessible. They protect against threats like copying models without permission, rebuilding sensitive training data, and turning on hidden malicious features. This section provides a comprehensive overview of strategies designed to protect LLMs at their core.

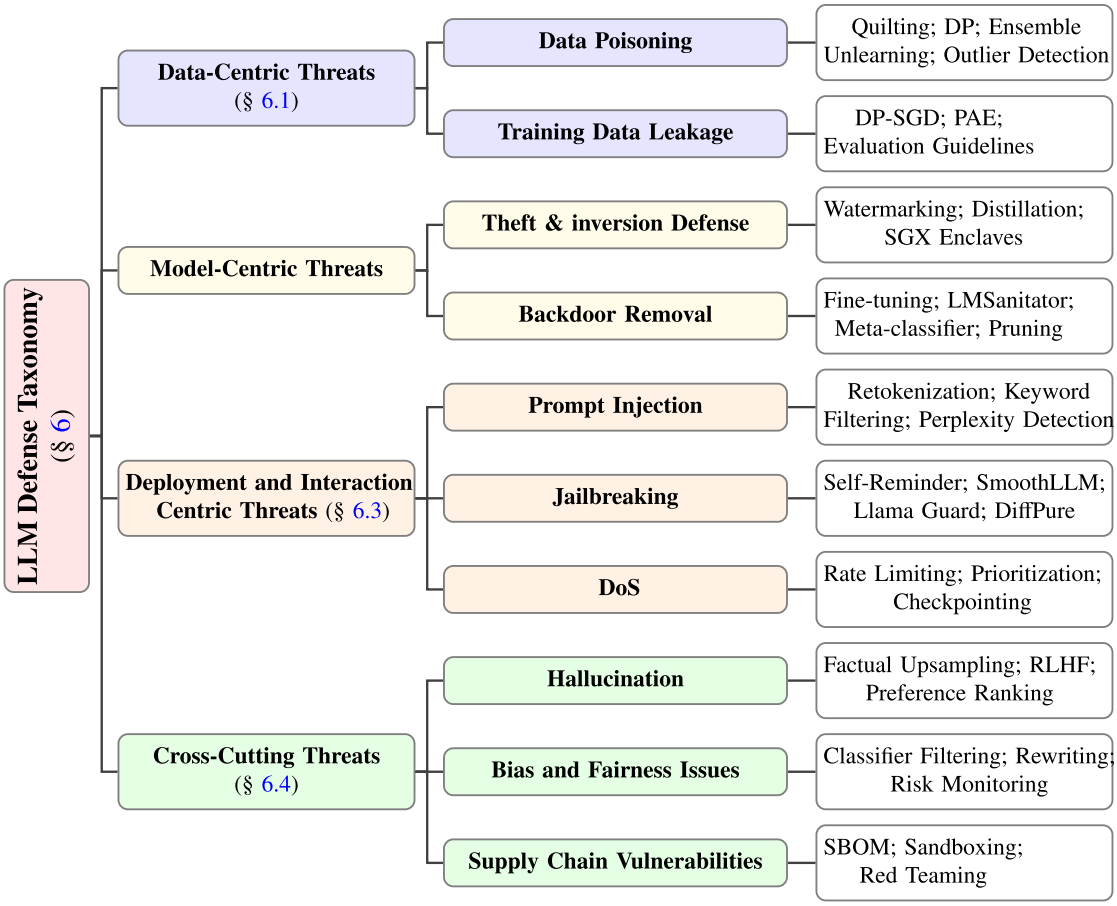


Fig. 11. Taxonomy of defense and mitigation strategies against LLM threats.

6.2.1. Defenses against model theft and inversion

A key strategy to mitigate model theft includes the use of output perturbation and watermarking techniques. By subtly modifying LLM outputs with an embedded signal or watermark, model proprietors can prevent unauthorized duplication and track occurrences of illicit model utilization. Methods proposed by Kirchenbauer et al. [150] for LLM watermarking incorporate imperceptible patterns in generated text, enabling detection for ownership verification or distribution tracking. These methods seek to achieve a balance between stealth and resilience against removal attacks. In addition to watermarking, API rate limiting and query monitoring serve as essential operational defenses. Establishing stringent rate limits on API requests and consistently analyzing query behaviors can facilitate the identification and prevention of automated model extraction efforts. Anomalous query behaviors, including a significant volume of varied queries from a single source, may indicate the presence of an ongoing attack [74].

A further effective strategy involves knowledge distillation and model hardening. Organizations can train smaller, more efficient models by distilling knowledge from a larger, proprietary LLM, rather than deploying the full model directly [151]. The distilled model, although not an exact replica of the original, effectively minimizes the attack surface for extraction while preserving satisfactory performance for designated tasks. This method inherently provides a level of protection since the complete, valuable model is not directly exposed. The implementation of secure enclaves and confidential computing provides a strong, hardware-supported layer of protection. Technologies such as Intel SGX and AMD SEV enable the execution of LLM parameters and inference logic within a cryptographically isolated environment, thereby preventing unauthorized access or tampering, even from compromised host systems [152]. This guarantees the protection of the model's internal state and proprietary knowledge throughout the computation process.

6.2.2. Defense against backdoor attack

Backdoor injection attacks are a big problem because malicious actors add hidden triggers to a large language model while it is being trained or fine-tuned. The model only acts badly when it gets certain, often subtle, types of input, and it works normally in other cases [85]. Finding and stopping these hidden attacks is very hard.

Sha et al. [153] showed that fine-tuning is a widely used and effective method for removing backdoors from machine learning models without significantly compromising their utility. Recognizing that independent fine-tuning could increase privacy risks for individual members, they introduced 'super fine-tuning.' Their experiments indicated that this super fine-tuning approach further reduced the risk of member data leakage. Expanding on the work of Liu et al. [82], several studies have presented alternative solutions for backdoor removal. Li et al. [154] introduced a knowledge distillation-based approach, where a fine-tuned teacher model assists the original model in concentrating on clean sample features to eliminate backdoors. For task-agnostic backdoors within language models, Wei et al. [155] developed a unique detection and removal technique. Their method involved reverse engineering to identify abnormal output features, targeting specific attack vectors rather than directly reversing the trigger token. Garcia-soto et al. [156] explored the differential robustness of clean and backdoored models to particular perturbations, utilizing these responses as distinguishing signatures to train a meta-classifier for backdoor identification. Another strategy, put forth by Shen et al. [157], involved reversing trigger tokens for specific labels by defining a convex hull in the input space and optimizing embedding vector coefficients through temperature scaling.

Additionally, research [158] shows that combining pruning with fine-tuning is a promising way to protect against backdoors. Tests of these combined methods have shown that they can effectively eliminate or mitigate backdoors in models.

6.3. Deployment and interaction-centric defenses

Once an LLM is deployed, the main point of attack changes from the training data and model parameters to the interface for live interaction. Deployment and interaction-centric defenses are meant to keep the LLM safe while it's running. They protect against threats that come from user inputs and API usage, like prompt injection, jailbreaking, and DoS attacks. These defenses form an important outer layer that ensures the model works safely and reliably in the real world.

6.3.1. Defense against prompt injection

The exploration of effective defense strategies against prompt injection attacks on LLMs is an important but still new area of research. Some studies systematically present existing defense mechanisms [104]. We can divide these defenses into two main groups: prevention-based and detection-based techniques. Specialized methods are also often used to deal with specific types of attacks, such as jailbreaking and P2SQL injection.

Prevention-based strategies are proactive measures designed to stop a harmful injected task from running successfully [159]. The main idea is to modify the input prompt before the model processes it, which will eliminate the threat. Paraphrasing and re-tokenization are two important ways to stop adversarial prompts. Both of these methods try to break the chain of injected instructions and stop the attack [159]. Data prompt isolation and instructional prevention are two other ways to stop problems before they happen [160]. Pre-processing inputs to find and block "red-flagged" keywords [161] that break usage rules, like those set by OpenAI [112], can serve as a simple defense. Kumar et al. also suggested using a safety filter on parts of input prompts. This method is certifiably strong, but it gets harder to use as the prompts get longer [162]. To protect against certain attacks, such as P2SQL injection, which can alter chatbots [163], it is effective to establish strict roles and permissions for databases. This strategy limits the execution of undesired SQL statements. Frameworks such as LangChain [163] could incorporate this method.

Detection-based defenses, in contrast, are focused on determining the integrity of a given data prompt [159,164,165]. One such method is perplexity-based detection, which operates on the premise that adding malicious instructions degrades a prompt's quality and increases its perplexity; a prompt is flagged as compromised if this value exceeds a set threshold [166]. Another approach, the proactive detection method, is to analyze the model's output [167]. This is effective in task-specific applications where the system anticipates a certain response format. If the generated output deviates from the proper answer, it implies a compromise [167].

6.3.2. Defense against jailbreaking attacks

Jailbreaking attacks, which use carefully crafted prompts to bypass safety features, have necessitated the development of specialized defenses. The system-mode self-reminder technique proposed by Wu et al., for instance, has been shown to drastically reduce jailbreaking success rates by having the system recall its core principles [168]. Similarly, Zhang et al. proposed a goal prioritization technique that prioritizes innocuous responses over useful ones during inference, thereby significantly reducing attack success [169]. For models such as Llama 2 and Vicuna [170], a very successful defense called SmoothLLM, which was proposed by Zou et al., effectively mitigates instruction-based attacks like AutoDAN [114], DAN [110], and PAIR [171] by perturbing a single input to create multiple copies and then aggregating the outputs to determine a final response. Additional specialized tools have also emerged, like Meta's Llama Guard, an input-output security measure intended to counteract these attacks [172]. Nie et al. recently proposed DiffPure, a diffusion model-based countermeasure against visual jailbreaking examples, to counter multi-modal threats [173].

Despite these developments, an empirical study by Liu et al. found that many prevalent prevention and detection techniques are insufficient against advanced, optimization-based attacks like JudgeDeceiver [174,175]. A significant challenge remains in balancing effective defense with model utility, as strong defenses may impair the LLM's performance [176]. Furthermore, the efficacy of many of these defenses, such as self-reminder-based methods [177] against complex attacks like DeepInception [178], has not yet been fully evaluated, indicating that this remains an active and critical area of cybersecurity research.

6.3.3. Defense against DoS attacks

These attacks, which aim to exhaust computational resources and render services unavailable, pose significant challenges to the reliability and availability of LLM-driven platforms. Mitigating these attacks requires the deployment of layered and adaptive mitigation mechanisms that can operate efficiently under adversarial conditions.

One foundational strategy for mitigating DoS risks is rate limiting, which involves imposing a ceiling on the number of requests a user or client can submit over a defined time window. For instance, limiting access to 10 requests per minute per user can effectively prevent adversaries from flooding the inference API. This approach, widely adopted by platforms such as OpenAI and Hugging Face, ensures fairness while preserving resources for legitimate users [179,180]. Complementing rate limiting, context window restrictions play a vital role in reducing computational strain. By capping the number of tokens allowed per input, for example, setting a maximum of 2000 tokens per request for models like GPT-3 or GPT-4, the system guards against resource exhaustion from excessively long or recursive prompts. Transformer-based models exhibit inference latency and memory consumption that scale superlinearly with input length, underscoring the efficacy of this measure [181].

To further ensure system responsiveness and maintain differentiated service levels, dynamic request prioritization can be employed. This mechanism ranks incoming requests based on user authentication, subscription tier, and behavioral profiling. Requests from trusted or premium users are processed preferentially, while potentially malicious or anomalous requests are delayed or blocked. Cloud-native API management solutions from providers like Google and Microsoft implement such differentiated request handling to uphold service-level objectives during periods of degraded performance or attack [182].

Finally, in scenarios where attacks induce system crashes or unstable behavior, the incorporation of model checkpointing and rollback mechanisms proves essential. By periodically saving the model and application state, administrators can restore service functionality with minimal downtime following an incident. This not only reduces service disruption but also facilitates forensic analysis to trace and understand the nature of the attack vector.

6.4. Cross-cutting threat mitigation strategies

A multifaceted and layered defense strategy is essential to effectively mitigate the cross-cutting threats identified in LLMs, as analyzed in Section 5.4. The following presents a detailed categorization of mitigation strategies, specifically designed to target the root causes and attack vectors associated with the identified threats.

6.4.1. Hallucination mitigation

- Improving the Quality of the Data for Training: Data quality improvement aims to decrease disinformation and societal bias. Selecting high-quality data sources from factual fields is the first step. Starting with GPT-2, Radford et al. [26] emphasized the importance of scraping only expert-curated web pages. Manual filtration gave the dataset credibility. As pre-training corpora have grown enormously, human-led curation has become impractical. Practitioners

use academic articles, encyclopedias, and technical manuals as high-fidelity truth sources to compensate. This approach is shown by The Pile [216] and other "textbook-like" corpora [217,218]. These sites provide comprehensive factual content that can help LLMs produce more accurate and less hallucinatory results. In addition, adding credible documents to the training mixture steers the model's learned priors toward factuality. Empirical studies have demonstrated that upsampling factual data improves model answers. Additionally, LLaMA2 specifically upweights the most factual sources during pre-training [91].

Another important part of data filtering is deduplication, which removes redundant content that can skew training dynamics and promote hallucinatory patterns. Many LLM developers now employ heuristic filtering techniques to enforce quality constraints at scale, as manually auditing each entry in massive training datasets is impractical. These include rule-based filtering, model-based scoring, and hybrid systems that incorporate syntactic, semantic, and contextual signals [24,91,219,220]. SFT datasets are usually smaller, hence manual data cleaning is still possible and widely done. LIMA shows that quality and diversity improve model alignment more than data quantity [221]. Recent improvements expand data filtering to create synthetic datasets to address hallucination-related behaviors including sycophancy, the model's tendency to agree with user ideas regardless of fact. The truth value of a claim and the user's stance were explicitly dissociated in one dataset, enabling more robust model fine-tuning [222]. This technique directly counters user-aligned training data's inaccurate assertion reinforcement.

- Human Feedback: Reinforcement learning from human feedback (RLHF) has been shown to enhance the factual accuracy of LLMs. RLHF typically comprises two phases: the training of a reward model utilizing human feedback and the optimization of a LLM based on the feedback from the reward model. GPT-4 [27] develops a reward model utilizing synthetic data to mitigate hallucinations, significantly enhancing its accuracy on the TruthfulQA dataset [223]. Other LLM systems, such InstructGPT [224], ChatGPT [225], and LLaMA2-Chat [91], utilize RLHF to enhance their efficacy. Nonetheless, reward hacking may occur in RLHF, meaning that the learnt reward model and human preferences do not consistently align [226]. LLaVA-RLHF [226] introduces Factually Augmented RLHF to enhance the reward model with real data. Furthermore, it is important to acknowledge that the implementation of RLHF algorithms is complex because to their complicated training processes and inconsistent performance [227]. To address this issue, researchers suggest acquiring human preferences in an offline method, where these preferences are articulated by ranking information [228–230] or natural language [231–233] to be incorporated into the SFT procedure.

6.4.2. Biased and toxic data mitigations

In a manner similar to techniques used to protect privacy-sensitive data, interventions targeting toxic and biased content aim to remove harmful or discriminatory material from large-scale web-sourced datasets in order to improve the overall quality of training data. Prior studies have focused on using annotated datasets to train classifiers capable of detecting toxic language [234]. For example, widely adopted tools such as the Perspective API [235] and Azure AI Content Safety [236] have been developed to automatically flag and filter harmful content within training corpora.

In the context of bias mitigation, the majority of strategies concentrate on recognizing and altering biased linguistic patterns. This encompasses tactics such as replacing biased terminology (gendered expressions) with neutral equivalents [237] or explicitly rewriting biased text segments with more inclusive alternatives [238]. These strategies seek to diminish prejudices and discriminatory associations in model outputs. However, new research indicates that these interventions possess certain disadvantages. A study by [239] indicates that excessively simplistic filtering methods can inadvertently elevate model training loss and may disproportionately omit content pertinent to particular demographic groups, thereby introducing novel biases. As a result, researchers have explored a range of techniques to address these challenges more effectively. For example, GPT-4 adopted a proactive data filtering approach to reduce toxicity and bias in its training data, whereas LLaMA chose not to apply such filtering techniques [27,91].

6.4.3. Mitigation of supply chain threats

Effective mitigation strategies are crucial to address the many vulnerabilities outlined in Section 5.4.3. Firstly, thorough evaluation of third-party components and data sources is essential. Given the substantial risks associated with dependency poisoning and compromised pre-trained models, organizations should exclusively utilize components from reliable and verified sources, as emphasized by OWASP standards [194]. The procedure entails examining the source code of open-source libraries and models when feasible, as well as validating the integrity and authenticity of datasets and models to ensure they remain unaltered during transit or storage [193,240]. A Software Bill of Materials (SBOM) is essential for maintaining a current inventory of all dependencies, including datasets and models, facilitating the rapid discovery of known vulnerabilities [241]. Secondly, the LLM ecosystem needs safe access restrictions and configurations. Enforcing fine-grained access permissions to sensitive supply chain data and limiting functionality to what is necessary reduces the attack surface. Proper code reviews and vulnerability scans are crucial for preventing path traversal and unsafe deserialization (e.g., CVE-2023-48299 in TorchServe) [193]. Furthermore, to reduce the impact of a compromised component on the system, consider sandboxing or containerizing third-party code [193]. Finally, proactive security auditing and incident response are essential. Performing regular security audits and vulnerability assessments, both internally and with key suppliers, helps uncover vulnerabilities before attackers can exploit them [193] To improve model resilience, undertake "red teaming" exercises to simulate attacks like prompt injection and data exfiltration [242]. A well-tested incident response plan helps companies limit breach damage and recover fast, minimizing financial and reputational loss. Today's interconnected software ecosystem requires multiple mitigation measures, from basic security to advanced threat detection, to protect LLM deployments' integrity and trustworthiness.

6.5. Cross-threat critical synthesis: Effectiveness, gaps, and persistent vulnerabilities

While Sections 5 and 6 systematically categorize threats and corresponding defenses across the LLM lifecycle, a comparative analysis reveals substantial asymmetries in defense maturity and effectiveness across threat classes. In particular, several core vulnerabilities remain only partially mitigated or fundamentally unresolved.

6.5.1. Data-centric threats

Although techniques such as dataset filtering, anomaly detection, and differential privacy reduce memorization and leakage risks, they do not fully address stealthy data poisoning or long-tail memorization effects [183,184]. Poisoning attacks remain difficult to detect at scale due to the opacity of large pretraining corpora and the lack of ground-truth data provenance [185]. Moreover, privacy-preserving techniques often introduce performance trade-offs, limiting their adoption in production-scale LLMs. As a result, data poisoning and training data leakage remain structurally persistent threats rather than fully solvable problems.

6.5.2. Model-centric threats

Watermarking, access control, and query monitoring can raise the cost of model theft, but they do not prevent high-fidelity imitation attacks under realistic black-box access assumptions [186]. Similarly, defenses against model inversion and backdoor injection rely heavily on assumptions about attacker knowledge or training-time controls that may not hold for foundation models trained by third parties[187,188].

These threats are therefore mitigable in controlled settings but remain unresolved in open or API-based deployment environments.

6.5.3. Deployment and interaction-centric threats

Particularly prompt injection and jailbreaking, represent the most dynamic and adversarially adaptive class. Despite extensive guardrail mechanisms, prompt filtering, and instruction consistency checks, empirical evidence consistently shows that new attack variants rapidly bypass static defenses [189]. This reflects a fundamental tension between instruction-following capability and robustness: improvements in controllability often expand the attack surface for semantic manipulation. Consequently, prompt injection is better characterized as an arms-race problem than a threat with a stable defensive solution [190].

6.5.4. Cross-cutting threats

Hallucinations, bias, and supply chain vulnerabilities, expose fundamental and systemic limitations of current LLM safety mechanisms. Existing hallucination mitigation approaches such as retrieval-augmented generation, confidence calibration, and self-verification can reduce the frequency of factual errors, but they do not provide formal guarantees of correctness or faithfulness, particularly in open-domain or reasoning-intensive tasks [191]. Similarly, bias mitigation techniques, including data curation, debiasing objectives, and post hoc filtering, remain highly context-dependent and often redistribute or obscure representational harms rather than eliminating them, especially across intersectional or underrepresented groups [192]. Supply chain vulnerabilities (spanning training datasets and third-party plugins) pose additional risks that are largely orthogonal to model-internal defenses. These threats are difficult to address through technical mechanisms alone and instead necessitate complementary governance measures, including dataset transparency, provenance tracking, security auditing, and regulatory oversight [193,194].

Overall, this synthesis suggests that while incremental defenses improve robustness in narrow settings, many LLM security and privacy threats persist due to structural factors: scale, opacity of training data, third-party dependencies, and the inherent flexibility of natural language interfaces. These findings motivate a shift from isolated countermeasures toward layered defenses, continuous evaluation, and lifecycle-aware risk management, which we further elaborate in Section 7.

7. Open challenges and future directions

7.1. Open challenges

Despite the significant progress made in LLMs, a numerous of critical challenges persist throughout the LLM lifecycle. This section summarizes the most important open challenges identified in recent literature.

7.1.1. Data ownership uncertainty

Unlike traditional ML models, LLMs are trained on extensive, largely uncurated internet corpora that may inadvertently include PII, proprietary documents, or sensitive content, often collected without the explicit consent of users. This practice raises significant concerns regarding data provenance, legal compliance, and ethical responsibility, particularly in light of regulatory frameworks such as the GDPR [243]. Moreover, the lack of formal recognition or rewards for data contributors diminishes incentives to supply high-quality data, thereby jeopardizing the sustainability of open-access data repositories that serve as foundational resources for training LLMs [244].

7.1.2. Memorization risks

Another major challenge lies in the tendency of LLMs to memorize and regurgitate sensitive training data. Even when data is anonymized using heuristic filters, such as regular expressions (regex), studies by Liu et al. [245] and others have shown that sensitive content can still be retrieved through carefully crafted prompts, model inversion attacks, or membership inference queries. These risks are compounded as LLMs scale in size, since larger models tend to exhibit greater memorization capacity, making privacy vulnerabilities even more acute. Moreover, machine unlearning, an emerging response to "right to be forgotten" mandates, remains immature [246]. Current approaches often fail to ensure complete erasure, degrade model performance, or inadvertently compromise unrelated knowledge, especially in complex, multi-modal models [17,247,248]. This highlight the need for more comprehensive evaluations and methodologies to enhance the effectiveness and efficiency of machine unlearning in LLMs.

7.1.3. Expanded attack surface

With the rapid deployment of LLMs and the widespread integration of third-party plugins, these models have become increasingly ubiquitous. As discussed in Section 5.4.3, such integrations enhance functionality and increase operational autonomy, they also significantly expand the attack surface [8]. This, in turn, introduces new potential vulnerabilities, not only within the models themselves but also in the external plugins [249]. LLMs remain susceptible to adversarial prompts, where seemingly benign inputs can elicit harmful or unauthorized outputs. This risk is further amplified in multi-modal or plugin-rich environments, where the complexity and openness of external interfaces create additional vectors for exploitation [250]. In addition, ensuring robustness in real-world scenarios requires treating security not as an afterthought or quick fix, but as a foundational design principle integrated throughout the model's lifecycle.

7.1.4. Security fragility in real-world scenarios

LLMs are increasingly being integrated into real-world applications. However, their widespread deployment has exposed critical security vulnerabilities [104]. Unlike controlled synthetic testing scenarios, real-world settings are far more dynamic, diverse, and unpredictable. As a result, security assumptions that hold in benchmark scenarios often fail to generalize, making it difficult to anticipate how LLMs will behave under novel or adversarial conditions.

7.1.5. Security vs. performance

Improving LLM security often comes with complexity and cost. For example, techniques like adversarial training and RLHF help reduce harmful or unsafe outputs. But they can also impair the model's performance on routine tasks. This trade-off is known as the "alignment tax". This term denotes the additional computational resources, training constraints, or performance sacrifices required to make an LLM behave in a safer, more ethically aligned manner. Many efforts have been developed to address this issue, like direct policy optimization (DPO) [228], self-play fine-tuning (SPIN) [251], and odds ratio preference optimization (ORPO) [252]. However, striking a sustainable balance between safety/alignment and performance remains an open and pressing research challenge, one that is critical for deploying LLMs responsibly in real-world environments.

7.1.6. Evaluation gaps

While LLMs have garnered significant attention for their broad applicability, evaluating their performance under real-world operational constraints remains a pressing and open challenge. Most existing benchmarks are static, narrowly focused on specific threats categories, and susceptible to overfitting, thereby offering only a partial angle on model safety, robustness, and generalization capabilities [22,244,253,254]. In addition, evaluation frameworks, like Alpaca-Eval [255] and Arena-Hard [256], often fail to capture the complexities of real-world deployments, including dynamic and multi-turn interactions, evolving adversarial tactics, and multimodal inputs [257]. Furthermore, they commonly overlook subtle yet impactful failure modes, such as hallucination, bias amplification, and performance degradation under adversarial or high-stress conditions. Consequently, reliance on such limited benchmarks may result in a misleading perception of system reliability. To address these limitations, there is a critical need for more comprehensive

and adaptive evaluation methodologies that can systematically probe LLM behavior across diverse, unpredictable, and safety-critical scenarios.

7.1.7. Interpretability and trust

Despite the strong performance of LLMs, their opaque, black-box nature presents a fundamental barrier to secure and trustworthy deployment, particularly in high-stakes domains such as security, healthcare, finance, and autonomous systems [258,259]. Interpretability and transparency refer to understanding a model's output and the accessibility of its internal processes [258,260]. In large-scale LLMs, both are significantly limited due to vast parameter spaces, emergent behaviors, and complex contextual dependencies, which obscure how outputs are generated [261]. This hinders efforts to trace vulnerabilities, identify root causes of adversarial behavior, and assess compliance with ethical or legal standards [262,263,263]. As LLMs are integrated into sensitive applications, such opacity raises critical concerns about accountability, trust, and risk management.

7.1.8. Accountability gap

LLMs trained on biased or unrepresentative data risk amplifying harmful stereotypes, generating toxic content, and reinforcing social inequalities [264]. These risks are compounded by the absence of clear accountability mechanisms. For example, when LLMs generate plagiarized or derivative content, questions of authorship and intellectual property emerge: who should be legally recognized as the creator? This ambiguity deepens when such content is published online and the question "Who created this content? becomes increasingly difficult to answer [244]. Likewise, when models generate misinformation or discriminatory statements, responsibility becomes unclear, whether it lies with the developers, deployers, or the model itself! The lack of defined accountability mechanisms raises significant ethical and legal concerns, particularly as LLMs are deployed in high-stakes domains such as healthcare, law, education, and finance.

7.2. Future directions

7.2.1. Lifelong adaptability

Static defense protocols, which depend on fixed adversarial datasets [265], are inadequate for the evolving threat landscape of real-world environments, where new threats continuously emerge. To ensure long-term security, future LLMs must embrace lifelong and continual learning frameworks that enable real-time adaptation, rather than relying on periodic retraining. These adaptive systems must integrate new information while preserving previously learned safety-critical behaviors, avoiding the pitfall of catastrophic forgetting [266]. At the same time, this adaptability must be achieved without incurring prohibitive computational costs, which remain a barrier to deployment. As such, future research should focus on designing efficient, incremental defense mechanisms that support real-time self-updating, maintain robust safety baselines, and scale sustainably with evolving threat landscapes.

7.2.2. Scalable privacy-preserving

Privacy remains a foundational challenge for LLMs, particularly due to their capacity to memorize and inadvertently leak sensitive training data. As models scale to trillions of parameters, future research must focus on developing scalable privacy-preserving mechanisms that maintain both security and utility. Federated Learning (FL) presents a promising solution, for mitigate memorization issues [267], by enabling LLMs to train on decentralized data across distributed devices, without requiring raw data to leave local environments [268–270]. This paradigm is especially suited for future edge-based LLM deployments, where data privacy and autonomy are paramount. However, FL alone does not guarantee formal privacy. To address this, Differential Privacy (DP) must be integrated to ensure that individual user data cannot be reverse-engineered, even if model gradients are intercepted [271]. By introducing noise into training signals, DP eliminates the need for a trusted central server and strengthens protection against inference attacks. Looking ahead, the fusion of FL and DP, along with advanced techniques like quantization and pruning [272,273] for reducing model size and computational cost, offers a compelling path forward [274]. This integrated approach enables the deployment of powerful, privacy-respecting LLMs at scale, especially in resource-constrained and privacy-sensitive edge environments.

7.2.3. Signature mechanisms

As LLMs gain widespread use, protecting their outputs and underlying intellectual property (IP) becomes crucial. To address this, future research should prioritize the development of robust signature mechanisms that can reliably attribute content and deter misuse. Two key directions for protecting LLM intellectual property are 1) watermarking [150] and 2) fingerprinting [275]. Watermarking embeds imperceptible, verifiable patterns into generated text to support ownership claims and traceability mitigating issues such as plagiarism, impersonation, and disinformation [75,150,276]. However, these techniques remain vulnerable to evasion through paraphrasing or token-level manipulation [277]. Fingerprinting, by contrast, identifies a model through its unique behavioral responses, such as reactions to specially crafted prompts, without altering the output itself [278]. While watermarking allows content-level attribution, fingerprinting enables model-level identification, together, these approaches provide a promising path for resilient IP protection in LLMs. Although still largely experimental, future work should explore hybrid schemes that integrate both methods to enhance resistance to evasion tactics, ensuring comprehensive IP protection across the entire LLM lifecycle while maintaining computational efficiency.

7.2.4. Explainability

The opaque decision-making processes of LLMs present serious challenges to security, accountability, and trust. These black-box models often exhibit emergent and unpredictable behaviors that are difficult to interpret, leaving room for vulnerabilities, exploitation, and misuse [279]. To address this, future research must prioritize the integration of XAI techniques into both the design and deployment phases of LLMs. XAI methods can illuminate the inner workings of LLMs by identifying influential input features and tracing decision paths, making it easier to detect anomalies, biases, or adversarial triggers. Yet, current explainability tools are insufficient for complex LLM architectures and often fail to support real-time, security-focused analysis. As such, new XAI frameworks must be model-agnostic, robust to adversarial manipulation, and tailored to language-specific behaviors. However, future efforts must carefully address the following challenges when deploying XAI techniques:

a) *Varying Stakeholder Needs:* Different stakeholders (e.g., developers, end-users, regulators) require different types of explanations, making it difficult to design a one-size-fits-all interpretability framework.
b) *Unified Evaluation Standards:* The absence of standardized benchmarks for evaluating XAI methods hinders meaningful comparison and adoption across systems [280].
c) *Compatibility:* Integrating XAI into current LLM infrastructures demands careful alignment to avoid architectural conflicts or deployment bottlenecks [281].
d) *Performance vs. Explainability:* Improving interpretability often comes at the expense of model performance or scalability, necessitating careful balancing.
e) *Complexity:* Additional explanatory components increase the system's complexity, potentially introducing maintenance burdens and latency.

f) *New Vulnerabilities:* Making models more transparent can inadvertently expose them to new forms of adversarial exploitation or information leakage [282].

Together, these considerations underscore the importance of multidisciplinary, risk-aware development of XAI frameworks for LLMs, not only to foster transparency and trust, but also to ensure security, robustness, and system scalability in real-world deployments.

8. Conclusion

LLMs are significantly transforming various applications. These applications have exposed several vulnerabilities stemming from the privacy and security challenges encountered by LLMs. This has garnered considerable attention from both academia and industry. The threats encountered by LLMs are significantly different from those faced by traditional models.

In this survey, we have provided a comprehensive and systematic analysis of the security and privacy landscape surrounding LLMs. As established in our introduction, the transformative potential of LLMs is matched by the complexity and severity of the vulnerabilities they introduce across their entire lifecycle. The existing research often fails to capture the interconnectedness of these risks, a gap this paper has sought to address.

Our primary contribution is examining the security issues that emerge with LLM usage, such as information leakage, unauthorized access, and insecure code generation. In addition, we explore various types of attacks that target LLMs and classify them into data-centric, model-centric, and deployment-centric categories, while also addressing cross-cutting issues like hallucinations and supply chain vulnerabilities. We have offered a more holistic perspective than found in prior literature. Furthermore, we have systematically mapped these threats to corresponding defense mechanisms, providing a critical evaluation of their effectiveness and limitations to guide practitioners in implementing layered security. As we move forward, the research community must focus on the open challenges identified, including the development of adaptive and scalable defenses, the creation of standardized evaluation frameworks and security tools, and the alignment of LLM systems with evolving regulatory requirements. This survey significantly helps researchers in comprehending the safety hazards posed by LLMs, hence facilitating their implementation across various domains.

CRediT authorship contribution statement

Aymen Dia Eddine Berini: Writing – review & editing, Writing – original draft, Visualization, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Norziana Jamil:** Writing – review & editing, Writing – original draft, Validation, Supervision, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization; **Ala-Eddine Benrazek:** Writing – review & editing, Writing – original draft, Validation, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Abderrahmane Lakas:** Writing – review & editing, Validation, Supervision, Project administration, Funding acquisition; **Leila Ismail:** Writing – review & editing, Validation, Project administration; **Mohamed Amine Ferrag:** Writing – review & editing, Validation; **Kwok-Yan Lam:** Validation, Supervision, Project administration.

Data availability

No data was used for the research described in the article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

Support from the UAE University for funding this work through Grant No. 12R316 is hereby gratefully acknowledged. The authors also acknowledge the support of the Big Data Analytics Center (BIDAC), UAE University.

References

[1] Z.Z. Chen, J. Ma, X. Zhang, N. Hao, A. Yan, A. Nourbakhsh, X. Yang, J. McAuley, L. Petzold, W.Y. Wang, A survey on large language models for critical societal domains: finance, healthcare, and law, arXiv preprint arXiv:2405.01769 (2024).
[2] Y. Chen, M. Cui, D. Wang, Y. Cao, P. Yang, B. Jiang, Z. Lu, B. Liu, A survey of large language models for cyber threat detection, Comput. Secur. 145 (2024) 104016.
[3] V. Rathod, S. Nabavirazavi, S. Zad, S.S. Iyengar, Privacy and security challenges in large language models, in: IEEE CCWC, IEEE, 2025, pp. 1–6.
[4] H. Debar, S. Dietrich, P. Laskov, E.C. Lupu, E. Ntoutsi, Emerging security challenges of large language models, preprint arXiv:arXiv:2412.17614 (2024).
[5] A. Purpura, S. Wadhwa, J. Zymet, A. Gupta, A. Luo, M.K. Rad, S. Shinde, M.S. Sorower, Building safe GenAI applications: an end-to-end overview of red teaming for large language models, arXiv preprint arXiv:2503.01742 (2025).
[6] K. Chen, X. Zhou, Y. Lin, S. Feng, L. Shen, P. Wu, A survey on privacy risks and protection in large language models, arXiv preprint arXiv:2505.01976 (2025).
[7] L. Ruhländer, E. Popp, M. Stylidou, S. Khan, D. Svetinovic, On the security and privacy implications of large language models: in-depth threat analysis, in: 2024 IEEE International Conferences on Internet of Things (iThings) and IEEE Green Computing & Communications (GreenCom) and IEEE Cyber, Physical & Social Computing (CPSCom) and IEEE Smart Data (SmartData) and IEEE Congress on Cybermatics, IEEE, 2024, pp. 543–550.
[8] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, Y. Zhang, A survey on large language model (LLM) security and privacy: the good, the bad, and the ugly, High Confid. Comput. 4 (2) (2024) 100211. https://doi.org/10.1016/j.hcc.2024.100211
[9] A.S. Inamdar, S. Eswaran, A comprehensive review of security and privacy issues in large language models, SSRN Electronic Journal (2024) https://doi.org/10.2139/ssrn.4825655.
[10] S. Wang, T. Zhu, B. Liu, M. Ding, X. Guo, D. Ye, W. Zhou, P.S. Yu, Unique security and privacy threats of large language model: A comprehensive survey, arXiv preprint arXiv:2406.07973 (2024).
[11] H. Li, Y. Chen, J. Luo, J. Wang, H. Peng, Y. Kang, X. Zhang, Q. Hu, C. Chan, Z. Xu, B. Hooi, Y. Song, Privacy in large language models: attacks, defenses and future directions, J. ACM 37 (4) (2024) Article 111.
[12] X. Huang, W. Ruan, W. Huang, G. Jin, Y. Dong, C. Wu, S. Bensalem, R. Mu, Y. Qi, X. Zhao, et al., A survey of safety and trustworthiness of large language models through the lens of verification and validation, Artif. Intell. Rev. 57 (7) (2024) 175.
[13] Y. Dong, R. Mu, Y. Zhang, S. Sun, T. Zhang, C. Wu, G. Jin, Y. Qi, J. Hu, J. Meng, et al., Safeguarding large language models: a survey, arXiv preprint arXiv:2406.02622 (2024).
[14] Y. Liu, Y. Yao, J.-F. Ton, X. Zhang, R. Guo, H. Cheng, Y. Klochkov, M.F. Taufiq, H. Li, Trustworthy LLMS: a survey and guideline for evaluating large language models' alignment, arXiv preprint arXiv:2308.05374 (2023).
[15] L. Weidinger, J. Uesato, M. Rauh, C. Griffin, P.-S. Huang, J. Mellor, A. Glaese, M. Cheng, B. Balle, A. Kasirzadeh, et al., Taxonomy of risks posed by language models, in: Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency, 2022, pp. 214–229.
[16] E. Shayegani, et al., Survey of vulnerabilities in large language models revealed by adversarial attacks, arXiv preprint arXiv:2310.10844 (2023).
[17] S. Abdali, R. Anarfi, C.J. Barberan, J. He, Securing large language models: threats, vulnerabilities and responsible practices, arXiv preprint arXiv:2403.12503 (2024).
[18] O. Friha, M.A. Ferrag, B. Kantarci, B. Cakmak, A. Ozgun, N. Ghoualmi-Zine, LLM-based edge intelligence: a comprehensive survey on architectures, applications, security and trustworthiness, IEEE Open J. Commun. Soc. 5 (2024) 5799–5856 https://doi.org/10.1109/OJCOMS.2024.3456549.
[19] A.G. Chowdhury, M.M. Islam, V. Kumar, F.H. Shezan, V. Jain, A. Chadha, Breaking down the defenses: a comparative survey of attacks on large language models, arXiv:2403.04786 (2024).
[20] B. Yan, K. Li, M. Xu, Y. Dong, Y. Zhang, Z. Ren, X. Cheng, On protecting the data privacy of large language models (LLMs): a survey, arXiv:2403.05156 (2024).
[21] B.C. Das, M.H. Amini, Y. Wu, Security and privacy challenges of large language models: a survey, ACM Comput. Surv. 57 (6) (2025) 1–39.
[22] P. Kumar, S. Mishra, Robustness in large language models: a survey of mitigation strategies and evaluation metrics, arXiv:2505.18658 (2025).
[23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention Is all you need, 2023, https://arxiv.org/abs/1706.03762.
[24] T.B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D.M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, 2020, https://arxiv.org/abs/2005.14165.
[25] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), 2019, pp. 4171–4186.

[26] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al., Language models are unsupervised multitask learners, OpenAI Blog 1 (8) (2019) 9.
[27] GPT-4 technical report, 2024, https://arxiv.org/abs/2303.08774.
[28] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, G. Lample, LLaMA: open and efficient foundation language models, 2023, https://arxiv.org/abs/2302.13971.
[29] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L.A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J.W. Rae, O. Vinyals, L. Sifre, Training compute-optimal large language models, 2022, https://arxiv.org/abs/2203.15556.
[30] R. Sennrich, B. Haddow, A. Birch, Neural machine translation of rare words with subword units, arXiv:1508.07909 (2015).
[31] T. Kudo, J. Richardson, Sentencepiece: a simple and language independent subword tokenizer and detokenizer for neural text processing, arXiv:1808.06226 (2018).
[32] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R.R. Salakhutdinov, Q.V. Le, XLNet: generalized autoregressive pretraining for language understanding, Proceedings of the 33rd International Conference on Neural Information Processing Systems 32 (2019) Curran Associates Inc. Red Hook, NY, USA 517 11.
[33] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al., Training language models to follow instructions with human feedback, Adv. Neural Inf. Process. Syst. 35 (2022) 27730–27744.
[34] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, S.R. Bowman, GLUE: a multi-task benchmark and analysis platform for natural language understanding, arXiv:1804.07461 (2018).
[35] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, J. Steinhardt, Measuring massive multitask language understanding, arXiv:2009.03300 (2020).
[36] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar, et al., Holistic evaluation of language models, arXiv:2211.09110 (2022).
[37] K. Singhal, S. Azizi, T. Tu, S.S. Mahdavi, J. Wei, H.W. Chung, N. Scales, A. Tanwani, H. Cole-Lewis, S. Pfohl, et al., Large language models encode clinical knowledge, Nature 620 (7972) (2023) 172–180.
[38] R. Luo, L. Sun, Y. Xia, T. Qin, S. Zhang, H. Poon, T.-Y. Liu, BioGPT: generative pre-trained transformer for biomedical text generation and mining, Brief. Bioinform. 23 (6) (2022) bbac409.
[39] S. Wu, O. Irsoy, S. Lu, V. Dabravolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, G. Mann, BloombergGPT: a large language model for finance, arXiv:2303.17564 (2023).
[40] B. Nelson, M. Barreno, F.J. Chi, A.D. Joseph, B.I.P. Rubinstein, U. Saini, C. Sutton, J.D. Tygar, K. Xia, Exploiting machine learning to subvert your spam filter, LEET 8 (1-9) (2008) 16–17.
[41] N. Carlini, M. Jagielski, C.A. Choquette-Choo, D. Paleka, W. Pearce, H. Anderson, A. Terzis, K. Thomas, F. Tramèr, Poisoning web-scale training datasets is practical, in: 2024 IEEE Symposium on Security and Privacy (SP), IEEE, 2024, pp. 407–425.
[42] A. Wan, E. Wallace, S. Shen, D. Klein, Poisoning language models during instruction tuning, in: International Conference on Machine Learning, PMLR, 2023, pp. 35413–35425.
[43] M. Shu, J. Wang, C. Zhu, J. Geiping, C. Xiao, T. Goldstein, On the exploitability of instruction tuning, Adv. Neural Inf. Process. Syst. 36 (2023) 61836–61856.
[44] X. Zhang, Z. Zhang, S. Ji, T. Wang, Trojaning language models for fun and profit, in: 2021 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, 2021, pp. 179–197.
[45] H. Aghakhani, W. Dai, A. Manoel, X. Fernandes, A. Kharkar, C. Kruegel, G. Vigna, D. Evans, B. Zorn, R. Sim, TrojanPuzzle: covertly poisoning code-suggestion models, in: 2024 IEEE Symposium on Security and Privacy (SP), IEEE, 2024, pp. 1122–1140.
[46] R. Schuster, C. Song, E. Tromer, V. Shmatikov, You autocomplete me: poisoning vulnerabilities in neural code completion, in: 30th USENIX Security Symposium (USENIX Security 21), 2021, pp. 1559–1575.
[47] S. Yan, S. Wang, Y. Duan, H. Hong, K. Lee, D. Kim, Y. Hong, An {LLM-assisted}{Easy-to-Trigger} backdoor attack on code completion models: injecting disguised vulnerabilities against strong detection, in: 33rd USENIX Security Symposium (USENIX Security 24), 2024, pp. 1795–1812.
[48] C. Mansfield, A. Paullada, K. Howell, Behind the mask: demographic bias in name detection for PII masking, arXiv:2205.04505 (2022).
[49] S. Zhao, Y. Yang, Z. Wang, Z. He, L.K. Qiu, L. Qiu, Retrieval augmented generation (RAG) and beyond: a comprehensive survey on how to make your LLMs use external data more wisely, arXiv:2409.14924 (2024).
[50] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, Q. Li, A survey on rag meeting LLMs: towards retrieval-augmented large language models, in: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 6491–6501.
[51] Z. Chen, Z. Xiang, C. Xiao, D. Song, B. Li, AgentPoison: red-teaming LLM agents via poisoning memory or knowledge bases, Adv. Neural Inf. Process. Syst. 37 (2024) 130185–130213.
[52] Y. Xu, J. Yao, M. Shu, Y. Sun, Z. Wu, N. Yu, T. Goldstein, F. Huang, ShadowCast: stealthy data poisoning attacks against vision-language models, arXiv:2402.06659 (2024).
[53] S. Shan, W. Ding, J. Passananti, S. Wu, H. Zheng, B.Y. Zhao, NightShade: prompt-specific poisoning attacks on text-to-image generative models, in: 2024 IEEE Symposium on Security and Privacy (SP), IEEE, 2024, pp. 807–825.
[54] J. Frankenfield, How personally identifiable information (PII) works, 2023, https://www.investopedia.com/terms/p/personally-identifiable-information-pii.asp.
[55] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, et al., Extracting training data from large language models, in: 30th USENIX Security Symposium (USENIX Security 21), 2021, pp. 2633–2650.
[56] OpenAI, March 20, chatGPT outage: Here's what happened, 2023, https://openai.com/index/march-20-chatgpt-outage/.
[57] C. Song, et al., Membership inference attacks against generative models, in: IEEE Symposium on Security and Privacy (S&P), 2020, pp. 1–15.
[58] N. Carlini, et al., Quantifying memorization across neural language models, in: Proceedings of the 40th International Conference on Machine Learning (ICML), 2023, pp. 1–20.
[59] L. Song, P. Mittal, Towards achieving user-level privacy in federated learning, in: NDSS, 2021, pp. 1–15.
[60] N. Lukas, A. Salem, R. Sim, S. Tople, L. Wutschitz, S. Zanella-Béguelin, Analyzing leakage of personally identifiable information in language models, in: 2023 IEEE Symposium on Security and Privacy (SP), IEEE, 2023, pp. 346–363.
[61] S. Zanella-Béguelin, et al., Leakage and forgetting in language models, in: IEEE Symposium on Security and Privacy, 2023, pp. 1–18.
[62] facebookresearch, GitHub - facebookresearch/metaseq: repo for external large-scale work, 2024, https://github.com/facebookresearch/metaseq.
[63] S. Kim, S. Yun, H. Lee, M. Gubri, S. Yoon, S.J. Oh, ProPILE: probing privacy leakage in large language models, Adv. Neural Inf. Process. Syst. 36 (2023) 20750–20762.
[64] Y. Nakamura, S. Hanaoka, Y. Nomura, N. Hayashi, O. Abe, S. Yada, S. Wakamiya, E. Aramaki, KART: parameterization of privacy leakage scenarios from pre-trained language models, arXiv:2101.00036 (2020).
[65] K.K. Nakka, A. Frikha, R. Mendes, X. Jiang, X. Zhou, PII-compass: guiding LLM training data extraction prompts towards the target PII via grounding, arXiv:2407.02943 (2024).
[66] J. Huang, H. Shao, K.C.-C. Chang, Are large pre-trained language models leaking your personal information?, arXiv:2205.12628 (2022).
[67] A. De Wynter, X. Wang, A. Sokolov, Q. Gu, S.-Q. Chen, An evaluation on large language model outputs: discourse and memorization, Nat. Lang. Process. J. 4 (2023) 100024.
[68] S. Biderman, U. Prashanth, L. Sutawika, H. Schoelkopf, Q. Anthony, S. Purohit, E. Raff, Emergent and predictable memorization in large language models, Adv. Neural Inf. Process. Syst. 36 (2023) 28072–28090.
[69] M. Nasr, N. Carlini, J. Hayase, M. Jagielski, A.F. Cooper, D. Ippolito, C.A. Choquette-Choo, E. Wallace, F. Tramèr, K. Lee, Scalable extraction of training data from (production) language models, arXiv:2311.17035 (2023).
[70] M. Abadi, A. Chu, I. Goodfellow, H.B. McMahan, I. Mironov, K. Talwar, L. Zhang, Deep learning with differential privacy, in: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, 2016, pp. 308–318.
[71] J. Borkar, What can we learn from data leakage and unlearning for law?, arXiv:2307.10476 (2023).
[72] L. Birch, W. Hackett, S. Trawicki, N. Suri, P. Garraghan, Model leeching: an extraction attack targeting LLMs, arXiv:2309.10544 (2023).
[73] F. Tramèr, F. Zhang, A. Juels, M.K. Reiter, T. Ristenpart, Stealing machine learning models via prediction {APIs}, in: 25th USENIX Security Symposium (USENIX Security 16), 2016, pp. 601–618.
[74] E. Wallace, M. Stern, D. Song, Imitation attacks and defenses for black-box machine translation systems, arXiv:2004.15015 (2020).
[75] W. Peng, J. Yi, F. Wu, S. Wu, B. Zhu, L. Lyu, B. Jiao, T. Xu, G. Sun, X. Xie, Are you copying my model? Protecting the copyright of large language models for eaas via backdoor watermark, arXiv:2305.10036 (2023).
[76] W.M. Si, M. Backes, Y. Zhang, Mondrian: prompt abstraction attack against large language models for cheaper API pricing, arXiv:2308.03558 (2023).
[77] M. Fredrikson, S. Jha, T. Ristenpart, Model inversion attacks that exploit confidence information and basic countermeasures, in: Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, 2015, pp. 1322–1333.
[78] L. Zhu, Z. Liu, S. Han, Deep leakage from gradients, in: Advances in Neural Information Processing Systems, 32, 2019.
[79] J. Geiping, H. Bauermeister, H. Dröge, M. Moeller, Inverting gradients–How easy is it to break privacy in federated learning?, in: Advances in Neural Information Processing Systems, 33, 2020, pp. 16937–16947.
[80] Y. Li, H. Huang, Y. Zhao, X. Ma, J. Sun, BackdoorLLM: a comprehensive benchmark for backdoor attacks on large language models, arXiv:2408.12798 (2024).
[81] W. Cai, et al., BadPrompt: backdoor attacks in continuous prompts, arXiv:2302.12173 (2023).
[82] X. Liu, et al., P-tuning V2: prompt tuning can be comparable to fine-tuning universally across scales and tasks, in: NeurIPS, 2021, pp. 1–12.
[83] N. Zhang, L. Li, X. Chen, S. Deng, Z. Bi, C. Tan, F. Huang, H. Chen, Differentiable prompt makes pre-trained language models better few-shot learners, arXiv:2108.13161 (2021).
[84] S. Zhao, J. Wen, L.A. Tuan, J. Zhao, J. Fu, Prompt as triggers for backdoor attack: examining the vulnerability in language models, arXiv:2305.01219 (2023).
[85] S. Zhao, M. Jia, Z. Guo, L. Gan, X. Xu, X. Wu, J. Fu, Y. Feng, F. Pan, L.A. Tuan, A Survey of Recent Backdoor Attacks and Defenses in Large Language Models, Trans. act. mach. learn. res. (2025) 2835–8856 https://openreview.net/forum?id=wZLWuFHxt5 Survey Certification.
[86] W. You, Z. Hammoudeh, D. Lowd, Large language models are better adversaries: exploring generative clean-label backdoor attacks against text classifiers, arXiv:2310.18603 (2023).
[87] J. Shi, Y. Liu, P. Zhou, L. Sun, BadGPT: exploring security vulnerabilities of chatgpt via backdoor attacks to instructGPT, arXiv:2304.12298 (2023).
[88] Y. Wang, D. Xue, S. Zhang, S. Qian, BadAgent: inserting and activating backdoor attacks in LLM agents, arXiv:2406.03007 (2024).

[89] J. Li, Y. Yang, Z. Wu, V.G. Vydiswaran, C. Xiao, ChatGPT as an attack tool: stealthy textual backdoor attack via blackbox generative model trigger, arXiv:2304.14475 (2023).

[90] H. Huang, Z. Zhao, M. Backes, Y. Shen, Y. Zhang, Composite backdoor attacks against large language models, arXiv:2310.07676 (2023).

[91] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al., LLaMA 2: open foundation and fine-tuned chat models, arXiv:2307.09288 (2023).

[92] J. He, G. Hou, X. Jia, Y. Chen, W. Liao, Y. Zhou, R. Zhou, Data stealing attacks against large language models via backdooring, Electronics 13 (14) (2024) 2858.

[93] Z. Feng, et al., CodeBERT: a pre-trained model for programming and natural languages, arXiv:2002.08155 (2020).

[94] Y. Wang, W. Wang, S. Joty, S.C.H. Hoi, Codet5: identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, arXiv:2109.00859 (2021).

[95] Z. Yang, B. Xu, J.M. Zhang, H.J. Kang, J. Shi, J. He, D. Lo, Stealthy backdoor attack for code models, IEEE Trans. Softw. Eng. 50 (4) (2024) 721–741.

[96] N. Kandpal, M. Jagielski, F. Tramèr, N. Carlini, Backdoor attacks for in-context learning with language models, arXiv:2307.14692 (2023).

[97] Y. Huang, T.Y. Zhuo, Q. Xu, H. Hu, X. Yuan, C. Chen, Training-free lexical backdoor attacks on language models, in: Proceedings of the ACM Web Conference 2023, 2023, pp. 2198–2208.

[98] Y. Li, T. Li, K. Chen, J. Zhang, S. Liu, W. Wang, T. Zhang, Y. Liu, Badedit: backdooring large language models by model editing, arXiv:2403.13355 (2024).

[99] A. Mittal, Prompt hacking and misuse of LLMs, 2023, (https://www.unite.ai/prompt-hacking-and-misuse-of-llm/). Accessed: 2025-05-27.

[100] owasap, OWASP top 10 for large language model applications, 2023, https://owasp.org/www-project-top-10-for-large-language-modelapplications/. Accessed: 2025-05-27.

[101] Owasp, OWASP top 10 for LLM applications 2025, 2025, https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/. Accessed: 2025-05-27.

[102] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection, in: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, 2023, pp. 79–90.

[103] F. Perez, I. Ribeiro, Ignore previous prompt: attack techniques for language models, arXiv:2211.09527 (2022).

[104] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, et al., Prompt injection attack against LLM-integrated applications, arXiv:2306.05499 (2023).

[105] D. Kang, X. Li, I. Stoica, C. Guestrin, M. Zaharia, T. Hashimoto, Exploiting programmatic behavior of LLMs: dual-use through standard security attacks, in: 2024 IEEE Security and Privacy Workshops (SPW), IEEE, 2024, pp. 132–143.

[106] B. Greshake Tzovaras, et al., More than you've asked for: a comprehensive analysis of prompt injection attacks in LLM-augmented systems, arXiv:2302.12173 (2023).

[107] J. Wang, Z. Liu, K.H. Park, Z. Jiang, Z. Zheng, Z. Wu, M. Chen, C. Xiao, Adversarial demonstration attacks on large language models, arXiv:2305.14950 (2023).

[108] X. Liu, Z. Yu, Y. Zhang, N. Zhang, C. Xiao, Automatic and universal prompt injection attacks against large language models, arXiv:2403.04957 (2024).

[109] H. Li, D. Guo, W. Fan, M. Xu, J. Huang, F. Meng, Y. Song, Multi-step jailbreaking privacy attacks on chatgpt, arXiv:2304.05197 (2023).

[110] X. Shen, Z. Chen, M. Backes, Y. Shen, Y. Zhang, " Do anything now": characterizing and evaluating in-the-wild jailbreak prompts on large language models, in: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, 2024, pp. 1671–1685.

[111] A. Rao, S. Vashistha, A. Naik, S. Aditya, M. Choudhury, Tricking LLMs into disobedience: formalizing, analyzing, and detecting jailbreaks, arXiv:2305.14965 (2023).

[112] G. Deng, Y. Liu, Y. Li, K. Wang, Y. Zhang, Z. Li, H. Wang, T. Zhang, Y. Liu, MasterKey: automated jailbreak across multiple large language model chatbots, arXiv:2307.08715 (2023).

[113] J. Yu, X. Lin, Z. Yu, X. Xing, GPTFUZZER: red teaming large language models with auto-generated jailbreak prompts, https://arxiv.org/abs/2309.10253. (2023).

[114] X. Liu, N. Xu, M. Chen, C. Xiao, AutoDAN: generating stealthy jailbreak prompts on aligned large language models, https://arxiv.org/abs/2310.04451. (2023).

[115] T. Liu, Y. Zhang, Z. Zhao, Y. Dong, G. Meng, K. Chen, Making them ask and answer: jailbreaking large language models in few queries via disguise and reconstruction, in: 33rd USENIX Security Symposium (USENIX Security 24), 2024, pp. 4711–4728.

[116] P. Chao, A. Robey, E. Dobriban, H. Hassani, G.J. Pappas, E. Wong, Jailbreaking black box large language models in twenty queries, in: 2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), IEEE, 2025, pp. 23–42.

[117] P. Ding, J. Kuang, D. Ma, X. Cao, Y. Xian, J. Chen, S. Huang, A Wolf in Sheep's clothing: generalized nested jailbreak prompts can fool large language models easily, https://arxiv.org/abs/2311.08268. (2023).

[118] Y. Liu, X. He, M. Xiong, J. Fu, S. Deng, B. Hooi, FlipAttack: jailbreak LLMs via flipping, https://arxiv.org/abs/2410.02832. (2024).

[119] I. Shumailov, Y. Zhao, D. Bates, N. Papernot, R. Mullins, R. Anderson, Sponge examples: energy-latency attacks on neural networks, in: 2021 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, 2021, pp. 212–231.

[120] K. Gao, T. Pang, C. Du, Y. Yang, S.-T. Xia, M. Lin, Denial-of-service poisoning attacks on large language models, https://arxiv.org/abs/2410.10760. (2024).

[121] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, et al., A survey on hallucination in large language models: principles, taxonomy, challenges, and open questions, ACM Trans. Inf. Syst. 43 (2) (2025) 1–55.

[122] A. Pal, L.K. Umapathi, M. Sankarasubbu, Med-halt: medical domain hallucination test for large language models, https://arxiv.org/abs/2307.15343. (2023).

[123] W. Wang, Z. Ma, Z. Wang, C. Wu, W. Chen, X. Li, Y. Yuan, A survey of LLM-based agents in medicine: how far are we from baymax?, https://arxiv.org/abs/2502.11211. (2025).

[124] H. Kang, X.-Y. Liu, Deficiency of large language models in finance: an empirical examination of hallucination, https://arxiv.org/abs/2311.15548. (2023).

[125] Y. Chen, Q. Fu, Y. Yuan, Z. Wen, G. Fan, D. Liu, D. Zhang, Z. Li, Y. Xiao, Hallucination detection: robustly discerning reliable answers in large language models, in: Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, 2023, pp. 245–255.

[126] N. Chakraborty, M. Ornik, K. Driggs-Campbell, Hallucination detection in foundation models for decision-making: a flexible definition and review of the state of the art, ACM Comput. Surv. 57 7 (2025) 1–35.

[127] A. Abid, M. Farooqi, J. Zou, Large language models associate muslims with violence, Nat. Mach. Intell. 3 (6) (2021) 461–463.

[128] R. Taylor, M. Kardas, G. Cucurull, T. Scialom, A. Hartshorn, E. Saravia, A. Poulton, V. Kerkez, R. Stojnic, Galactica: a large language model for science, 2022, arXiv:2211.09085. https://arxiv.org/abs/2211.09085.

[129] S. Dai, Y. Zhou, L. Pang, W. Liu, X. Hu, Y. Liu, X. Zhang, G. Wang, J. Xu, Neural retrievers are biased towards LLM-generated content, in: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 526–537.

[130] D. Huang, J.M. Zhang, Q. Bu, X. Xie, J. Chen, H. Cui, Bias testing and mitigation in llm-based code generation, ACM Trans. Softw. Eng. Method. 35 1 (2025) 1–31.

[131] J. Dhamala, T. Sun, V. Kumar, S. Krishna, Y. Pruksachatkun, K.-W. Chang, R. Gupta, Bold: dataset and metrics for measuring biases in open-ended language generation, in: Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency, 2021, pp. 862–872.

[132] A. Tamkin, A. Askell, L. Lovitt, E. Durmus, N. Joseph, S. Kravec, K. Nguyen, J. Kaplan, D. Ganguli, Evaluating and mitigating discrimination in language model decisions, https://arxiv.org/abs/2312.03689. (2023).

[133] S. Wang, Y. Zhao, Z. Liu, Q. Zou, H. Wang, SoK: understanding vulnerabilities in the large language model supply chain, 2025, arXiv:2502.12497. https://arxiv.org/abs/2502.12497.

[134] J. He, M. Vechev, Large language models for code: security hardening and adversarial testing, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, 2023, pp. 1865–1879.

[135] Y. Nie, Y. Wang, J. Jia, M.J. De Lucia, N.D. Bastian, W. Guo, D. Song, TrojFM: resource-efficient backdoor attacks against very large foundation models, https://arxiv.org/abs/2405.16783. (2024).

[136] U. Iqbal, T. Kohno, F. Roesner, Llm platform security: applying a systematic evaluation framework to openai's chatgpt plugins, in: Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society, 7, 2024, pp. 611–623.

[137] N. Jyothi Swaroop Arlagadda, V.S.M. Vuppalapati, S. Modi, R. Vishwakarma, H. Shah, SpeechCraft: an integrated data generation pipeline from videos for LLM finetuning, 2024 29th International Conference on Automation and Computing (ICAC) (2024) 1–6. https://doi.org/10.1109/ICAC61394.2024.10718747

[138] C. Dwork, F. McSherry, K. Nissim, A. Smith, Calibrating noise to sensitivity in private data analysis, in: Theory of Cryptography Conference, Springer, 2006, pp. 265–284.

[139] Y. Ma, Y. Wang, Efficient distribution estimation for data with unobserved subpopulation identifiers, Electron. J. Stat. 6 (2012) 710.

[140] J. Lee, Y. Cho, R. Lee, S. Yuk, J. Youn, H. Park, D. Shin, A novel data sanitization method based on dynamic dataset partition and inspection against data poisoning attacks, Electronics 14 (2) (2025) 374.

[141] J. Xu, M.D. Ma, F. Wang, C. Xiao, M. Chen, Instructions as backdoors: backdoor vulnerabilities of instruction tuning for large language models, https://arxiv.org/abs/2305.14710. (2023).

[142] M. Goldblum, D. Tsipras, C. Xie, X. Chen, A. Schwarzschild, D. Song, A. Mądry, B. Li, T. Goldstein, Dataset security for machine learning: data poisoning, backdoor attacks, and defenses, IEEE Trans. Pattern Anal. Mach. Intell. 45 (2) (2022) 1563–1580.

[143] N. Baracaldo, B. Chen, H. Ludwig, J.A. Safavi, Mitigating poisoning attacks on machine learning models: a data provenance based approach, in: Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security, 2017, pp. 103–110.

[144] E. Wallace, T.Z. Zhao, S. Feng, S. Singh, Concealed data poisoning attacks on NLP models, https://arxiv.org/abs/2010.12563. (2020).

[145] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, et al., Siren's song in the AI ocean: a survey on hallucination in large language models, https://arxiv.org/abs/2309.01219. (2023).

[146] M. Bonchek, et al., Why the problem with learning is unlearning, Harv. Bus. Rev. 85 (3) (2016) 413–414.

[147] X. Xiao, Y. Zhang, J. Xu, W. Ren, J. Zhang, Assessment methods and protection strategies for data leakage risks in large language models, J. Ind. Eng. Appl. Sci. 3 (2) (2025) 6–15.

[148] D. Venditti, E.S. Ruzzetti, G.A. Xompero, C. Giannone, A. Favalli, R. Romagnoli, F.M. Zanzotto, Enhancing data privacy in large language models through private association editing, https://arxiv.org/abs/2406.18221. (2024).

[149] K. Zhou, Y. Zhu, Z. Chen, W. Chen, W.X. Zhao, X. Chen, Y. Lin, J.-R. Wen, J. Han, Don't make your LLM an evaluation benchmark cheater, https://arxiv.org/abs/2311.01964. (2023).

[150] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, T. Goldstein, A watermark for large language models, in: International Conference on Machine Learning, PMLR, 2023, pp. 17061–17084.

[151] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, https://arxiv.org/abs/1503.02531. (2015).

[152] S. Arnautov, B. Trach, F. Gregor, T. Knauth, A. Martin, C. Priebe, J. Lind, D. Muthukumaran, D. O'keeffe, M.L. Stillwell, et al., {SCONE}: secure linux containers with intel {SGX}, in: 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), 2016, pp. 689–703.

[153] Z. Sha, X. He, P. Berrang, M. Humbert, Y. Zhang, Fine-tuning is all you need to mitigate backdoor attacks, https://arxiv.org/abs/2212.09067. (2022).

[154] Y. Li, X. Lyu, N. Koren, L. Lyu, B. Li, X. Ma, Neural attention distillation: erasing backdoor triggers from deep neural networks, https://arxiv.org/abs/2101.05930. (2021).

[155] C. Wei, W. Meng, Z. Zhang, M. Chen, M. Zhao, W. Fang, L. Wang, Z. Zhang, W. Chen, LMSanitator: defending prompt-tuning against task-agnostic backdoors, https://arxiv.org/abs/2308.13904. (2023).

[156] D. Garcia-soto, H. Chen, F. Koushanfar, PerD: perturbation sensitivity-based neural trojan detection framework on NLP applications, https://arxiv.org/abs/2208.04943. (2022).

[157] G. Shen, Y. Liu, G. Tao, Q. Xu, Z. Zhang, S. An, S. Ma, X. Zhang, Constrained optimization with dynamic bound-scaling for effective NLP backdoor defense, in: International Conference on Machine Learning, PMLR, 2022, pp. 19879–19892.

[158] K. Liu, B. Dolan-Gavitt, S. Garg, Fine-pruning: defending against backdooring attacks on deep neural networks, in: International Symposium on Research in Attacks, Intrusions, and Defenses, Springer, 2018, pp. 273–294.

[159] N. Jain, A. Schwarzschild, Y. Wen, G. Somepalli, J. Kirchenbauer, P.-y. Chiang, M. Goldblum, A. Saha, J. Geiping, T. Goldstein, Baseline defenses for adversarial attacks against aligned language models, https://arxiv.org/abs/2309.00614. (2023).

[160] S. Schulhoff, Instruction defense, strengthen AI prompts against hacking, 2024, (https://learnprompting.org/docs/prompt_hacking/defensive_measures/instruction). Accessed: 2025-06-20.

[161] A. Wei, N. Haghtalab, J. Steinhardt, Jailbroken: how does LLM safety training fail?, Adv. Neural Inf. Process. Syst. 36 (2023) 80079–80110.

[162] A. Kumar, C. Agarwal, S. Srinivas, A.J. Li, S. Feizi, H. Lakkaraju, Certifying LLM safety against adversarial prompting, https://arxiv.org/abs/2309.02705. (2023).

[163] R. Pedro, D. Castro, P. Carreira, N. Santos, From prompt injections to SQL injection attacks: how protected is your LLM-integrated web application?, https://arxiv.org/abs/2308.01990. (2023).

[164] R. Socher, A. Perelygin, J. Wu, J. Chuang, C.D. Manning, A.Y. Ng, C. Potts, Recursive deep models for semantic compositionality over a sentiment treebank, in: Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, 2013, pp. 1631–1642.

[165] Y. Wang, J. Deng, A. Sun, X. Meng, Perplexity from PLM is unreliable for evaluating text quality, https://arxiv.org/abs/2210.05892. (2022).

[166] H. Gonen, S. Iyer, T. Blevins, N.A. Smith, L. Zettlemoyer, Demystifying prompts in language models via perplexity estimation, https://arxiv.org/abs/2212.04037. (2022).

[167] J. Selvi, Exploring prompt injection attacks, 2023, (https://www.nccgroup.com/us/research-blog/exploring-prompt-injection-attacks/). Accessed: 2025-06-26.

[168] Y. Xie, J. Yi, J. Shao, J. Curl, L. Lyu, Q. Chen, X. Xie, F. Wu, Defending chatGPT against jailbreak attack via self-reminders, Nat. Mach. Intell. 5 (12) (2023) 1486–1496.

[169] Z. Zhang, J. Yang, P. Ke, F. Mi, H. Wang, M. Huang, Defending large language models against jailbreaking attacks through goal prioritization, https://arxiv.org/abs/2311.09096. (2023).

[170] A. Zou, Z. Wang, N. Carlini, M. Nasr, J.Z. Kolter, M. Fredrikson, Universal and transferable adversarial attacks on aligned language models, https://arxiv.org/abs/2307.15043. (2023).

[171] P. Chao, A. Robey, E. Dobriban, H. Hassani, G.J. Pappas, E. Wong, Jailbreaking black box large language models in twenty queries, https://arxiv.org/abs/2310.08419. (2023).

[172] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, et al., LLaMA guard: LLM-based input-output safeguard for human-ai conversations, https://arxiv.org/abs/2312.06674. (2023).

[173] W. Nie, B. Guo, Y. Huang, C. Xiao, A. Vahdat, A. Anandkumar, Diffusion models for adversarial purification, https://arxiv.org/abs/2205.07460. (2022).

[174] Y. Liu, Y. Jia, R. Geng, J. Jia, N.Z. Gong, Formalizing and benchmarking prompt injection attacks and defenses, in: 33rd USENIX security symposium (USENIX security 24), 2024, pp. 1831–1847.

[175] J. Shi, Z. Yuan, Y. Liu, Y. Huang, P. Zhou, L. Sun, N.Z. Gong, Optimization-based prompt injection attack to LLM-as-a-judge, in: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, 2024, pp. 660–674.

[176] L. Li, T. Xie, B. Li, Sok: certified robustness for deep neural networks, in: 2023 IEEE Symposium on Security and Privacy (SP), IEEE, 2023, pp. 1289–1310.

[177] Y. Wu, Y. Huang, Y. Liu, X. Li, P. Zhou, L. Sun, Can large language models automatically jailbreak GPT-4V?, https://arxiv.org/abs/2407.16686. (2024).

[178] X. Li, Z. Zhou, J. Zhu, J. Yao, T. Liu, B. Han, Deepinception: hypnotize large language model to be jailbreaker, https://arxiv.org/abs/2311.03191. (2023).

[179] OpenAI, OpenAI Platform — platform.openai.com, 2025, https://platform.openai.com/docs/guides/rate-limits. [Accessed 26-06-2025].

[180] H. Face, Inference endpoints - hugging face — huggingface.co, 2025, (https://huggingface.co/inference-endpoints/dedicated). [Accessed 26-06-2025].

[181] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, J. Dean, Efficiently scaling transformer inference, Proc. Mach. Learn. Syst. 5 (2023) 606–624.

[182] dlepow, API management documentation — learn.microsoft.com, 2025, https://learn.microsoft.com/en-us/azure/api-management/. [Accessed 26-06-2025].

[183] A. Al-Kaswan, M. Izadi, A. van Deursen, Traces of memorisation in large language models for code, in: 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE) (2023) 944–955. https://doi.org/10.1145/3597503.3639133

[184] X. Xiao, Y. Zhang, H. Chen, W. Ren, J. Zhang, J. Xu, A differential privacy-based mechanism for preventing data leakage in large language model training, Acad. J. Sociol. Manage. 3 (2) (2025) 33–42.

[185] T. Fu, M. Sharma, P. Torr, S.B. Cohen, D. Krueger, F. Barez, Poisonbench: assessing large language model vulnerability to data poisoning, https://arxiv.org/abs/2410.08811. (2024).

[186] H. Jia, C.A. Choquette-Choo, V. Chandrasekaran, N. Papernot, Entangled watermarks as a defense against model extraction, in: 30th USENIX Security Symposium (USENIX Security 21), 2021, pp. 1937–1954.

[187] W. Ma, D. Wang, Y. Song, M. Xue, S. Wen, Z. Li, Y. Xiang, TrapNet: model inversion defense via trapdoor, IEEE Trans. Inf. Forensics Secur. 20 (2025) 4469–4483 https://doi.org/10.1109/TIFS.2025.3560557.

[188] Y. Chen, S. Shao, E. Huang, Y. Li, P.-Y. Chen, Z. Qin, K. Ren, Refine: inversion-free backdoor defense via model reprogramming, https://arxiv.org/abs/2502.18508. (2025).

[189] Y. Chen, H. Li, Z. Zheng, D. Wu, Y. Song, B. Hooi, Defense against prompt injection attack by leveraging attack techniques, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2025, pp. 18331–18347.

[190] T.-C. Liu, C.-Y. Hsu, K.-Y. Lee, C.-A. Fu, H.-y. Lee, AEGIS: automated co-evolutionary framework for guarding prompt injections schema, https://arxiv.org/abs/2509.00088. (2025).

[191] W. Li, W. Wu, M. Chen, J. Liu, X. Xiao, H. Wu, Faithfulness in natural language generation: a systematic survey of analysis, evaluation and optimization methods, https://arxiv.org/abs/2203.05227. (2022).

[192] E.M. Bender, T. Gebru, A. McMillan-Major, S. Shmitchell, On the dangers of stochastic parrots: can language models be too big?, in: Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency, 2021, pp. 610–623.

[193] A.K. Nandibhatla, OWASP LLM Top 10: addressing supply chain risks in AI systems, 2025, (https://scrumgit.com/supply-chain-risks-securing-ai-components-from-external-threats-dc6c04c3fc5f). Accessed: 2025-07-20.

[194] OWaSP, LLM03:2025 supply chain, 2025, https://genai.owasp.org/llmrisk/llm032025-supply-chain/. Accessed: 2025-07-20.

[195] N. Shahin, L. Ismail, ChatGPT, let us chat sign language: experiments, architectural elements, challenges and research directions, in: 2023 International Symposium on Networks, Computers and Communications (ISNCC), IEEE, 2023, pp. 1–7.

[196] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, et al., Improving language understanding by generative pre-training, 2018 https://www.bibsonomy.org/bibtex/273ced32c0d4588eb95b6986dc2c8147c/jonaskaiser.

[197] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, L. Zettlemoyer, BART: denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension, 2019, arXiv:1910.13461. https://arxiv.org/abs/1910.13461.

[198] openAI, Introducing GPT-5 — openai.com, 2025https://openai.com/index/introducing-gpt-5/,[Accessed 10-09-2025].

[199] M.A. Ferrag, F. Alwahedi, A. Battah, B. Cherif, A. Mechri, N. Tihanyi, T. Bisztray, M. Debbah, Generative AI in cybersecurity: A comprehensive review of LLM applications and vulnerabilities, Internet Things Cyber-Phys. Syst. 5 (2025) 1–46 https://www.sciencedirect.com/science/article/pii/S2667345225000082 https://doi.org/10.1016/j.iotcps.2025.01.001.

[200] F. Tambon, A. Moradi-Dakhel, A. Nikanjam, F. Khomh, M.C. Desmarais, G. Antoniol, Bugs in large language models generated code: an empirical study, Empirical Softw. Eng. 30 (3) (2025) 65.

[201] Z. Chu, S. Wang, J. Xie, T. Zhu, Y. Yan, J. Ye, A. Zhong, X. Hu, J. Liang, P.S. Yu, et al., LLM agents for education: advances and applications, https://arxiv.org/abs/2503.11733. (2025).

[202] S.R. Peddinti, S.R. Katragadda, B.K. Pandey, A. Tanikonda, Utilizing large language models for advanced service management: potential applications and operational challenges, J. Sci. Technol. 4 (2) (2023).

[203] S. Hassani, Enhancing legal compliance and regulation analysis with large language models, in: 2024 IEEE 32nd International Requirements Engineering Conference (RE), IEEE, 2024, pp. 507–511.

[204] F.V. Jedrzejewski, D. Fucci, O. Adamov, ThreMoLIA: threat modeling of large language model-integrated applications, ArXiv abs/2504.18369 (2025). https://doi.org/10.48550/arXiv.2504.18369

[205] N. Nagaraja, H. Bahsi, Cyber threat modeling of an LLM-based healthcare system, arXiv preprint (2024).

[206] S.B. Tete, Threat modelling and risk analysis for large language model (llm)-powered applications, https://arxiv.org/abs/2406.11007. (2024).

[207] F. Bayhan, Prompt injection attacks on large language models: a systematic literature review, Master's Thesis, Middle East Technical University (2025).

[208] A. Verma, S. Krishna, S. Gehrmann, M. Seshadri, A. Pradhan, T. Ault, L. Barrett, D. Rabinowitz, J. Doucette, N. Phan, Operationalizing a threat model for red-teaming large language models (LLMs), https://arxiv.org/abs/2407.14937. (2024).

[209] I. Elsharef, Z. Zeng, Z. Gu, Facilitating threat modeling by leveraging large language models, in: Workshop on AI Systems with Confidential Computing, 2024, pp. 1–6.

[210] F. Qi, Y. Chen, X. Zhang, M. Li, Z. Liu, M. Sun, Mind the style of text! adversarial and backdoor attacks based on text style transfer, https://arxiv.org/abs/2110.07139. (2021).

[211] X. Chen, et al., BadChain: backdoor chain-of-thought prompting for large language models, https://arxiv.org/abs/2308.XXXXX. (2023).

[212] J. Wei, X. Wang, D. Schuurmans, et al., Chain-of-thought prompting elicits reasoning in large language models, https://arxiv.org/abs/2201.11903. (2022).

[213] A. Esmradi, D.W. Yip, C.F. Chan, A comprehensive survey of attack techniques, implementation, and mitigation strategies in large language models, in: International Conference on Ubiquitous Security, Springer, 2023, pp. 76–95.

[214] S. Zanella-Beguelin, S. Tople, A. Paverd, B. Köpf, Grey-box extraction of natural language models, in: International Conference on Machine Learning, PMLR, 2021, pp. 12278–12286.

[215] M. Du, X. Yue, S.S.M. Chow, T. Wang, C. Huang, H. Sun, DP-forward: fine-tuning and inference on language models with differential privacy in forward pass, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, 2023, pp. 2665–2679.

[216] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, et al., The pile: an 800GB dataset of diverse text for language modeling, https://arxiv.org/abs/2101.00027. (2020).

[217] Y. Li, S. Bubeck, R. Eldan, A. Del Giorno, S. Gunasekar, Y.T. Lee, Textbooks are all you need II: phi-1.5 technical report, https://arxiv.org/abs/2309.05463. (2023).

[218] S. Gunasekar, Y. Zhang, J. Aneja, C.C.T. Mendes, A. Del Giorno, S. Gopi, M. Javaheripi, P. Kauffmann, G. de Rosa, O. Saarikivi, et al., Textbooks are all you need, https://arxiv.org/abs/2306.11644. (2023).

[219] G. Penedo, Q. Malartic, D. Hesslow, R. Cojocaru, A. Cappelli, H. Alobeidli, B. Pannier, E. Almazrouei, J. Launay, The RefinedWeb dataset for Falcon LLM: outperforming curated corpora with web data, and web data only, https://arxiv.org/abs/2306.01116. (2023).

[220] A. Abbas, K. Tirumala, D. Simig, S. Ganguli, A.S. Morcos, SemDeDup: data-efficient learning at web-scale through semantic deduplication, https://arxiv.org/abs/2303.09540. (2023).

[221] C. Zhou, P. Liu, P. Xu, S. Iyer, J. Sun, Y. Mao, X. Ma, A. Efrat, P. Yu, L. Yu, et al., Lima: less is more for alignment, Adv. Neural Inf. Process. Syst. 36 (2023) 55006–55021.

[222] J. Wei, D. Huang, Y. Lu, D. Zhou, Q.V. Le, Simple synthetic data reduces sycophancy in large language models, https://arxiv.org/abs/2308.03958. (2023).

[223] S. Lin, J. Hilton, O. Evans, TruthfulQA: measuring how models mimic human falsehoods, https://arxiv.org/abs/2109.07958. (2021).

[224] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C.L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, R. Lowe, Training language models to follow instructions with human feedback, 2022, arXiv:2203.02155. https://arxiv.org/abs/2203.02155.

[225] O. Blog, Introducing chatGPT, Internet: https://openai. com/blog/chatgpt 5 (2022).

[226] Z. Sun, S. Shen, S. Cao, H. Liu, C. Li, Y. Shen, C. Gan, L.-Y. Gui, Y.-X. Wang, Y. Yang, et al., Aligning large multimodal models with factually augmented RLHF, https://arxiv.org/abs/2309.14525. (2023).

[227] T. Shen, R. Jin, Y. Huang, C. Liu, W. Dong, Z. Guo, X. Wu, Y. Liu, D. Xiong, Large language model alignment: a survey, https://arxiv.org/abs/2309.15025. (2023).

[228] R. Rafailov, A. Sharma, E. Mitchell, C.D. Manning, S. Ermon, C. Finn, Direct preference optimization: your language model is secretly a reward model, Adv. Neural Inf. Process. Syst. 36 (2023) 53728–53741.

[229] F. Song, B. Yu, M. Li, H. Yu, F. Huang, Y. Li, H. Wang, Preference ranking optimization for human alignment, Proc. AAAI Conf. Artif. Intell., 38, 2024, pp. 18990–18998.

[230] Z. Yuan, H. Yuan, C. Tan, W. Wang, S. Huang, F. Huang, RRHF: rank responses to align language models with human feedback without tears, https://arxiv.org/abs/2304.05302. (2023).

[231] H. Liu, C. Sferrazza, P. Abbeel, Chain of hindsight aligns language models with feedback, https://arxiv.org/abs/2302.02676. (2023).

[232] R. Liu, C. Jia, G. Zhang, Z. Zhuang, T. Liu, S. Vosoughi, Second thoughts are best: learning to re-align with human values from text edits, Adv. Neural Inf. Process. Syst. 35 (2022) 181–196.

[233] R. Liu, R. Yang, C. Jia, G. Zhang, D. Zhou, A.M. Dai, D. Yang, S. Vosoughi, Training socially aligned language models on simulated social interactions, 2023, arXiv:2305.16960. https://arxiv.org/abs/2305.16960.

[234] S.V. Georgakopoulos, S.K. Tasoulis, A.G. Vrahatis, V.P. Plagianakos, Convolutional neural networks for toxic comment classification, in: Proceedings of the 10th Hellenic Conference on Artificial Intelligence, 2018, pp. 1–6.

[235] S. Schulhoff, Perspective API documentation, 2021, (https://github.com/conversationai/perspectiveapi). Accessed: 2025-07-20.

[236] A.I. Azure, Azure AI content safety, 2023, (https://azure.microsoft.com/en-us/products/ai-services/ai-content-safety). Accessed: 2025-07-20.

[237] J. Zhao, T. Wang, M. Yatskar, R. Cotterell, V. Ordonez, K.-W. Chang, Gender bias in contextualized word embeddings, https://arxiv.org/abs/1904.03310. (2019).

[238] R.H. Maudslay, H. Gonen, R. Cotterell, S. Teufel, It's all in the name: mitigating gender bias with name-based counterfactual data substitution, https://arxiv.org/abs/1909.00871. (2019).

[239] C. Wald, L. Pfahler, Exposing bias in online communities through large-scale language models, https://arxiv.org/abs/2306.02294. (2023).

[240] A. Mechri, M.A. Ferrag, M. Debbah, SecureQwen: leveraging LLMs for vulnerability detection in python codebases, Comput. Secur. 148 (2025) 104151.

[241] A. Linskens, The OWASP LLM top 10 and sonatype: supply chain security, 2025, https://www.sonatype.com/blog/the-owasp-llm-top-10-and-sonatype-supply-chain-security. Accessed: 2025-07-20.

[242] S. Moore, LLM security: top 10 risks and 7 security best practices, 2025, https://www.exabeam.com/explainers/ai-cyber-security/llm-security-top-10-risks-and-7-security-best-practices/Accessed: 2025-07-20.

[243] V. Paul, A. von dem Bussche, The EU General Data Protection Regulation (GDPR), Springer, 2024. https://doi.org/10.1007/978-3-031-62328-8

[244] X. Ma, Y. Gao, Y. Wang, R. Wang, X. Wang, Y. Sun, Y. Ding, H. Xu, Y. Chen, Y. Zhao, H. Huang, Y. Li, J. Zhang, X. Zheng, Y. Bai, Z. Wu, X. Qiu, J. Zhang, Y. Li, X. Han, H. Li, et al., Safety at scale: a comprehensive survey of large model safety, https://arxiv.org/abs/2502.05206. (2025).

[245] Y. Liu, J. Huang, Y. Li, D. Wang, B. Xiao, Generative AI model privacy: a survey, Artif. Intell. Rev. 58 (1) (2024) 33.

[246] Y. Xu, Machine unlearning for traditional models and large language models: a short survey, https://arxiv.org/abs/2404.01206. (2024).

[247] H. Xu, T. Zhu, L. Zhang, W. Zhou, W. Zhao, Toward efficient target-level machine unlearning based on essential graph, IEEE Trans. Neural Netw. Learn. Syst. 36 8 (2025) 14581–14595 https://doi.org/10.1109/TNNLS.2024.3514607.

[248] J. Cheng, H. Amiri, Multidelete for multimodal machine unlearning, in: European Conference on Computer Vision, Springer, 2024, pp. 165–184.

[249] OWASP Foundation, OWASP top 10 for large language model applications 2025, Technical Report, Open worldwide application security project, 2025. Version 2.0, https://owasp.org/www-project-top-10-for-large-language-model-applications/.

[250] H. Li, Y. Chen, J. Luo, J. Wang, H. Peng, Y. Kang, X. Zhang, Q. Hu, C. Chan, Z. Xu, et al., Privacy in large language models: attacks, defenses and future directions, https://arxiv.org/abs/2310.10383. (2023).

[251] Z. Chen, Y. Deng, H. Yuan, K. Ji, Q. Gu, Self-play fine-tuning converts weak language models to strong language models, https://arxiv.org/abs/2401.01335. (2024).

[252] J. Hong, N. Lee, J. Thorne, ORPO: monolithic preference optimization without reference model, https://arxiv.org/abs/2403.07691. (2024).

[253] Y. Tu, R. Hu, J. Sang, ODE: open-set evaluation of hallucinations in multimodal large language models, in: Proceedings of the Computer Vision and Pattern Recognition Conference, 2025, pp. 19836–19845.

[254] C. Zhu, N. Chen, Y. Gao, Y. Zhang, P. Tiwari, B. Wang, Is your LLM outdated? A deep look at temporal generalization, https://arxiv.org/abs/2405.08460. (2024).

[255] Y. Dubois, B. Galambosi, P. Liang, T.B. Hashimoto, Length-controlled alpacaeval: a simple way to debias automatic evaluators, https://arxiv.org/abs/2404.04475. (2024).

[256] T. Li, W.-L. Chiang, E. Frick, L. Dunlap, T. Wu, B. Zhu, J.E. Gonzalez, I. Stoica, From crowdsourced data to high-quality benchmarks: arena-hard and benchbuilder pipeline, https://arxiv.org/abs/2406.11939. (2024).

[257] R. Raju, S. Jain, B. Li, J. Li, U. Thakker, Constructing domain-specific evaluation sets for LLM-as-a-judge, https://arxiv.org/abs/2408.08808. (2024).

[258] Y. Huang, L. Sun, H. Wang, S. Wu, Q. Zhang, Y. Li, C. Gao, Y. Huang, W. Lyu, Y. Zhang, et al., TrustLLM: trustworthiness in large language models, https://arxiv.org/abs/2401.05561. (2024).

[259] T. Ali, P. Kostakos, HuntGPT: integrating machine learning-based anomaly detection and explainable ai with large language models (LLMs), https://arxiv.org/abs/2309.16021. (2023).

[260] Q.V. Liao, J.W. Vaughan, Ai transparency in the age of LLMs: a human-centered research roadmap, https://arxiv.org/abs/2306.01941. 10 (2023).

[261] Z. Tan, T. Chen, Z. Zhang, H. Liu, Sparsity-guided holistic explanation for llms with interpretable inference-time intervention, Proc. AAAI Conf. Artif. Intell., 38, 2024, pp. 21619–21627.

[262] A.B. Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. García, S. Gil-López, D. Molina, R. Benjamins, et al., Explainable artificial intelligence (XAI): concepts, taxonomies, opportunities and challenges toward responsible AI, Inf. Fusion 58 (2020) 82–115.

[263] M. Nyre-Yu, E. Morris, M. Smith, B. Moss, C. Smutz, Explainable AI in Cybersecurity Operations: Lessons Learned from xAI Tool Deployment, Technical Report, Sandia National Lab.(SNL-NM), Albuquerque, NM (United States), 2022.

[264] K. Wang, G. Zhang, Z. Zhou, J. Wu, M. Yu, S. Zhao, C. Yin, J. Fu, Y. Yan, H. Luo, L. Lin, Z. Xu, H. Lu, et al., A comprehensive survey in LLM(-agent) full stack safety: data, training and deployment, 2025, https://arxiv.org/abs/2504.15585.

[265] Z. Zhang, M. Fang, L. Chen, M.-R. Namazi-Rad, J. Wang, How do large language models capture the ever-changing world knowledge? A review of recent advances, https://arxiv.org/abs/2310.07343. (2023).

[266] J. Zheng, S. Qiu, C. Shi, Q. Ma, Towards lifelong learning of large language models: a survey, ACM Comput. Surv. 57 (8) (2025) 1–35.

[267] O.D. Thakkar, S. Ramaswamy, R. Mathews, F. Beaufays, Understanding unintended memorization in language models under federated learning, in: Proceedings of the Third Workshop on Privacy in Natural Language Processing, 2021, pp. 1–10.

[268] B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Artificial Intelligence and Statistics, PMLR, 2017, pp. 1273–1282.

[269] T. Fan, Y. Kang, G. Ma, W. Chen, W. Wei, L. Fan, Q. Yang, Fate-LLM: a industrial grade federated learning framework for large language models, https://arxiv.org/abs/2310.10049. (2023).

[270] H. Woisetschläger, A. Erben, S. Wang, R. Mayer, H.-A. Jacobsen, Federated fine-tuning of llms on the very edge: the good, the bad, the ugly, in: Proceedings of the Eighth Workshop on Data Management for End-to-End Machine Learning, 2024, pp. 39–50.

[271] M. Miranda, E.S. Ruzzetti, A. Santilli, F.M. Zanzotto, S. Bratières, E. Rodolà, Preserving privacy in large language models: a survey on current threats and solutions, https://arxiv.org/abs/2408.05212. (2024).

[272] M. Sun, Z. Liu, A. Bair, J.Z. Kolter, A simple and effective pruning approach for large language models, https://arxiv.org/abs/2306.11695. (2023).

[273] E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh, GPTQ: accurate post-training quantization for generative pre-trained transformers, https://arxiv.org/abs/2210.17323. (2022).

[274] Y. Youn, Z. Hu, J. Ziani, J. Abernethy, Randomized quantization is all you need for differential privacy in federated learning, https://arxiv.org/abs/2306.11913. (2023).
[275] A. Sarabi, T. Yin, M. Liu, An LLM-based framework for fingerprinting internet-connected devices, in: Proceedings of the 2023 ACM on Internet Measurement Conference, 2023, pp. 478–484.
[276] L. Tang, G. Uberti, T. Shlomi, Baselines for identifying watermarked large language models, https://arxiv.org/abs/2305.18456. (2023).
[277] V.S. Sadasivan, A. Kumar, S. Balasubramanian, W. Wang, S. Feizi, Can AI-generated text be reliably detected?, https://arxiv.org/abs/2303.11156. (2023).
[278] Y.-Y. Tsai, C. Guo, J. Yang, L. van der Maaten, RoFL: robust fingerprinting of language models, https://arxiv.org/abs/2505.12682. (2025).
[279] G. Rjoub, J. Bentahar, O.A. Wahab, R. Mizouni, A. Song, R. Cohen, H. Otrok, A. Mourad, A survey on explainable artificial intelligence for cybersecurity, IEEE Trans. Netw. Serv. Manage. 20 (4) (2023) 5115–5140.
[280] M. Nauta, J. Trienes, S. Pathak, E. Nguyen, M. Peters, Y. Schmitt, J. Schlötterer, M. Van Keulen, C. Seifert, From anecdotal evidence to quantitative evaluation methods: a systematic review on evaluating explainable ai, ACM Comput. Surv. 55 (13s) (2023) 1–42.
[281] G. Srivastava, R.H. Jhaveri, S. Bhattacharya, S. Pandya, P.K.R. Maddikunta, G. Yenduri, J.G. Hall, M. Alazab, T.R. Gadekallu, et al., XAI for cybersecurity: state of the art, challenges, open issues and future directions, (2022) arxiv:2206.03585
[282] N. Liu, M. Du, R. Guo, H. Liu, X. Hu, Adversarial attacks and defenses: an interpretation perspective, ACM SIGKDD Explorations Newsl. 23 (1) (2021) 86–99.